厦门大学信息学院 本科选修课
2024-2025 第二学期

模 式 识 别

Pattern Recognition

主 讲：王程教授、陈龙彪副教授

厦门大学信息学院计算机科学与技术系

人工智能引论101计划系列课程



Transformer

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

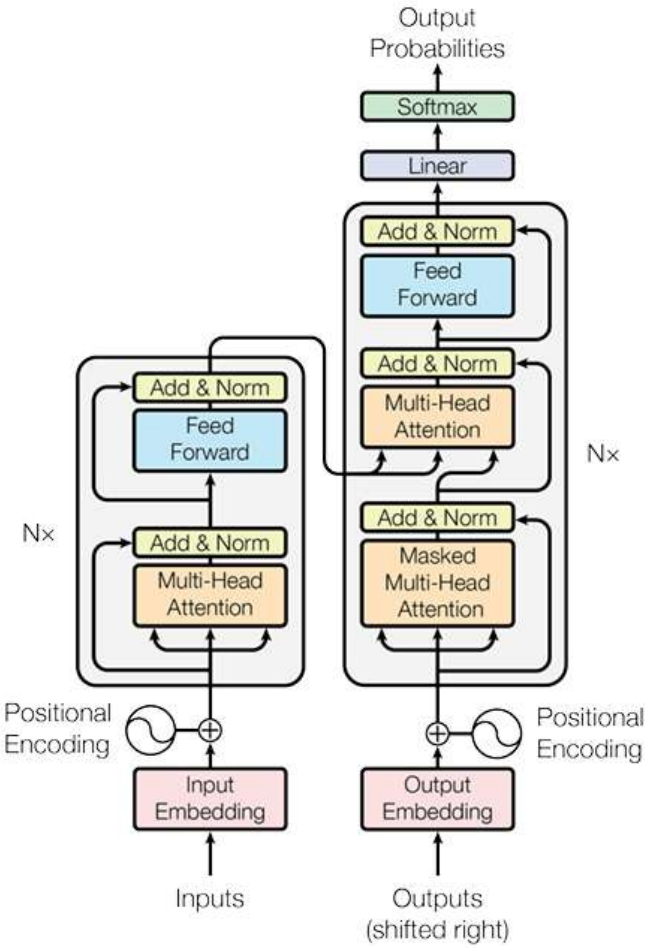
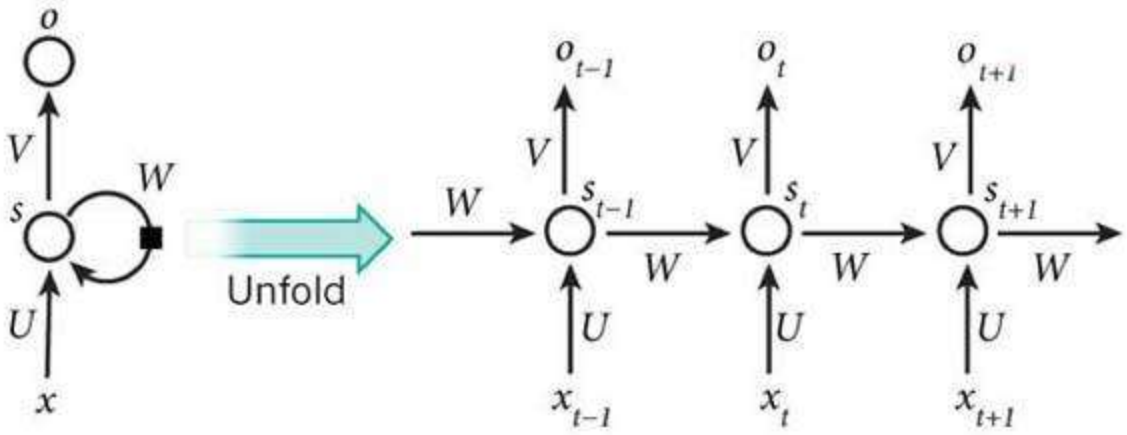


Figure 1: The Transformer - model architecture.

Transformer介绍

口 为什么需要Transformer

在没有transformer的时候，我们都是用什么来完成这系列的任务的呢?

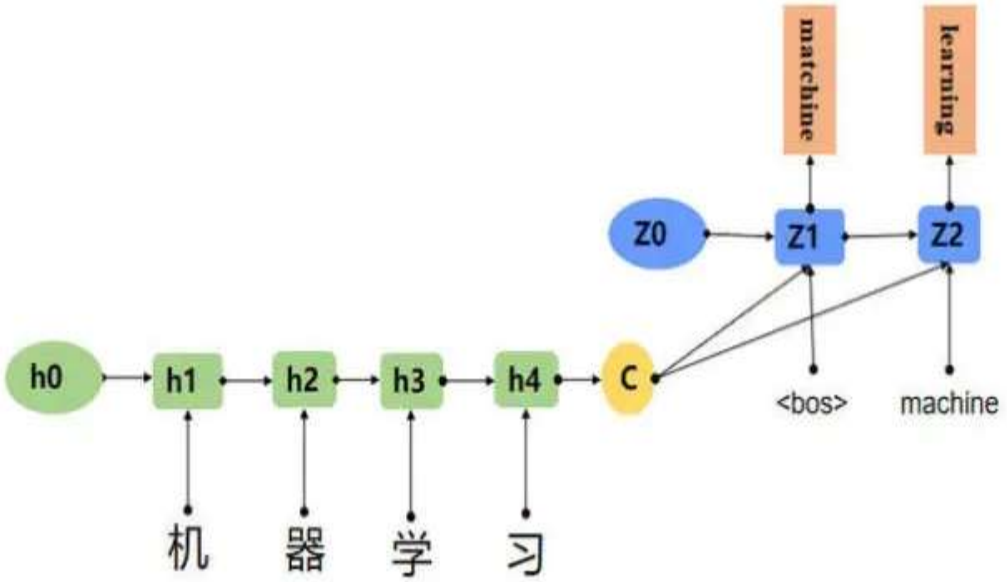


其实在之前我们使用的是RNN（或者是其的单向或者双向变种LSTM/GRU等） 来作为编解码器。RNN模块每次只能够吃进一个输入token和前一次的隐藏状态，然后得到输出。它的时序结构使得这个模型能够得到长距离的依赖关系，但是这也使得它**不能够并行计算**，模型效率十分低。

Transformer介绍

□ Seq2Seq任务

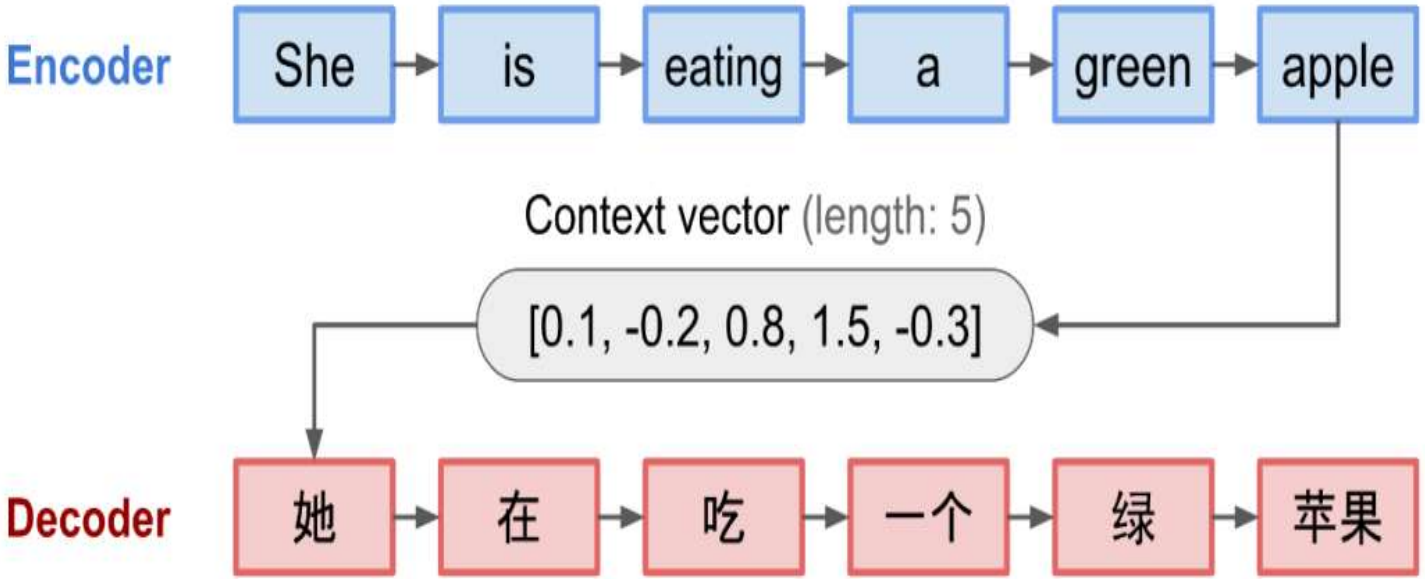
Seq2Seq 任务指的是输入和输出都是序列的任务，输出的长度不确定时采用的模型，这种情况一般是在机器翻译的任务中出现，将一句中文翻译成英文，那么这句英文的长度有可能会比中文短，也有可能会比中文长，所以输出的长度就不确定了。



Transformer介绍

□ Encoder-Decoder模型

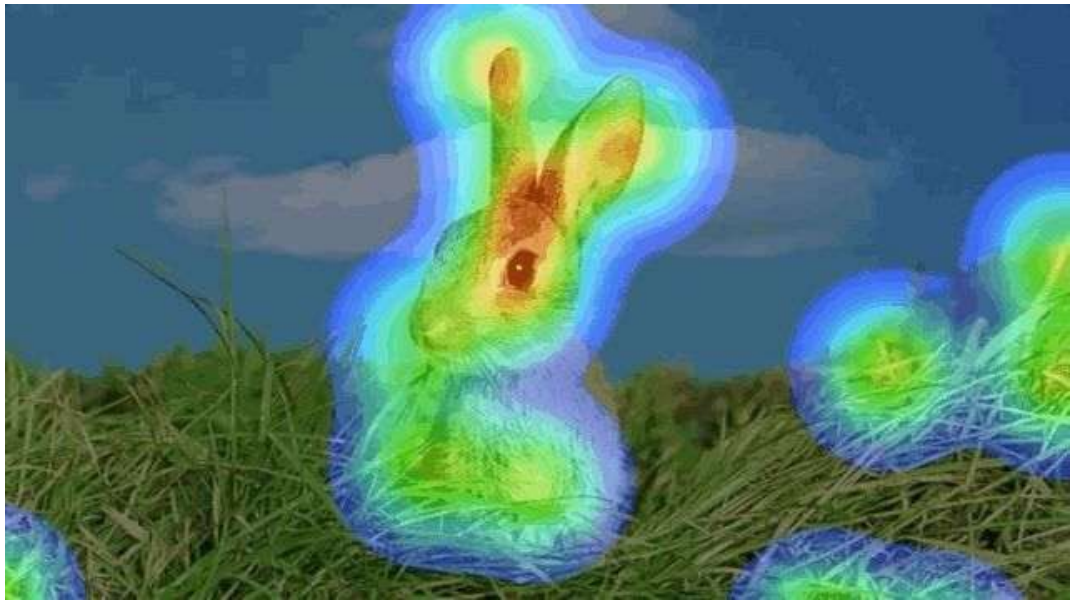
通常来说，Seq2Seq任务最常见的是使用Encoder+Decoder的模式，先将一个序列编码成一个上下文矩阵，在使用Decoder来解码。当然，我们仅仅把context vector作为编码器到解码器的输入。



Transformer介绍

□ Attention注意力机制

在介绍什么是注意力机制之前，先让大家看一张图片。当大家看到下面图片，会首先看到什么内容？当过载信息映入眼帘时，我们的大脑会把注意力放在主要的信息上，这就是大脑的注意力机制。



Transformer介绍

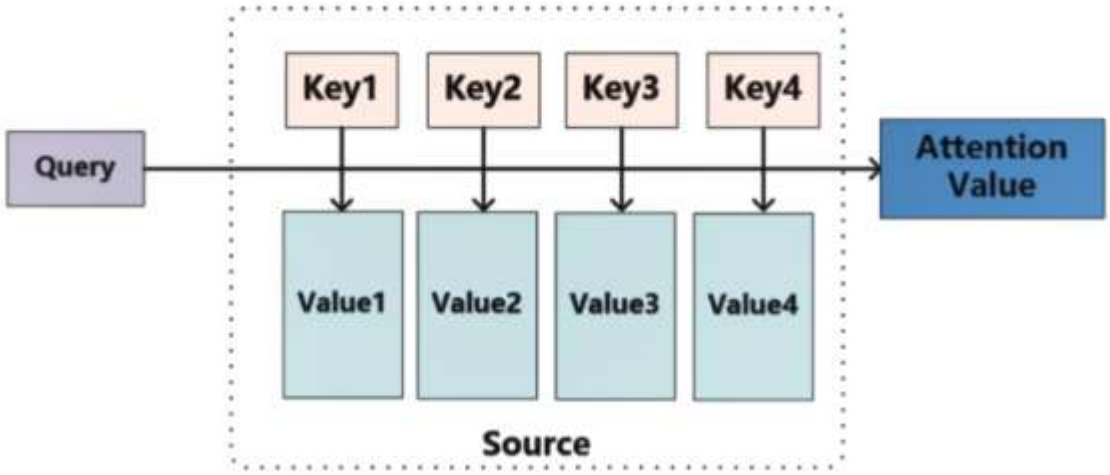
□ 每个词的Attention计算

每个词的Q会跟整个序列中每一个K计算得分，然后基于得分再分配特征

Q: query，要去查询的

K: key，等着被查的

V: value，实际的特征信息



Transformer介绍

□ Attention的优点

1.**参数少**：相比于 CNN、RNN ，其复杂度更小，参数也更少。所以对算力的要求也就更小。

2.**速度快**：Attention 解决了 RNN及其变体模型不能并行计算的问题。Attention机制每一步计算不依赖于上一步的计算结果，因此可以和CNN一样并行处理。

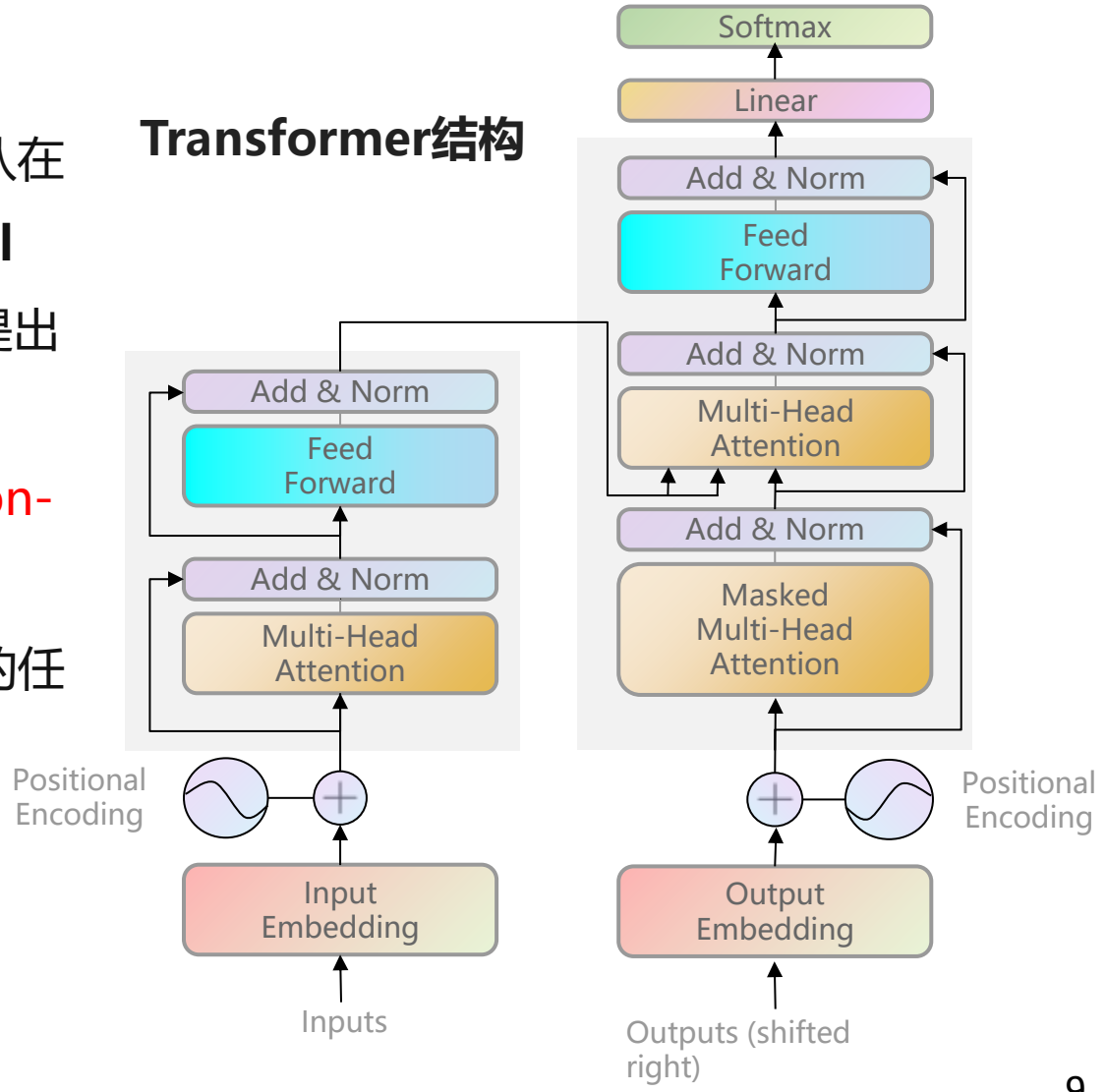
3.**效果好**：在Attention 机制引入之前，有一个问题大家一直很苦恼：长距离的信息会被弱化，就好像记忆能力弱的人，记不住过去的事情是一样的。

Transformer介绍

□ Transformer

2017年google的机器翻译团队在NIPS上发表了**Attention is all you need**的文章，开创性地提出了在序列转录领域，完全抛弃CNN和RNN，只依赖Attention-注意力结构的简单的网络架构，名为Transformer；论文实现的任务是机器翻译。

Transformer结构



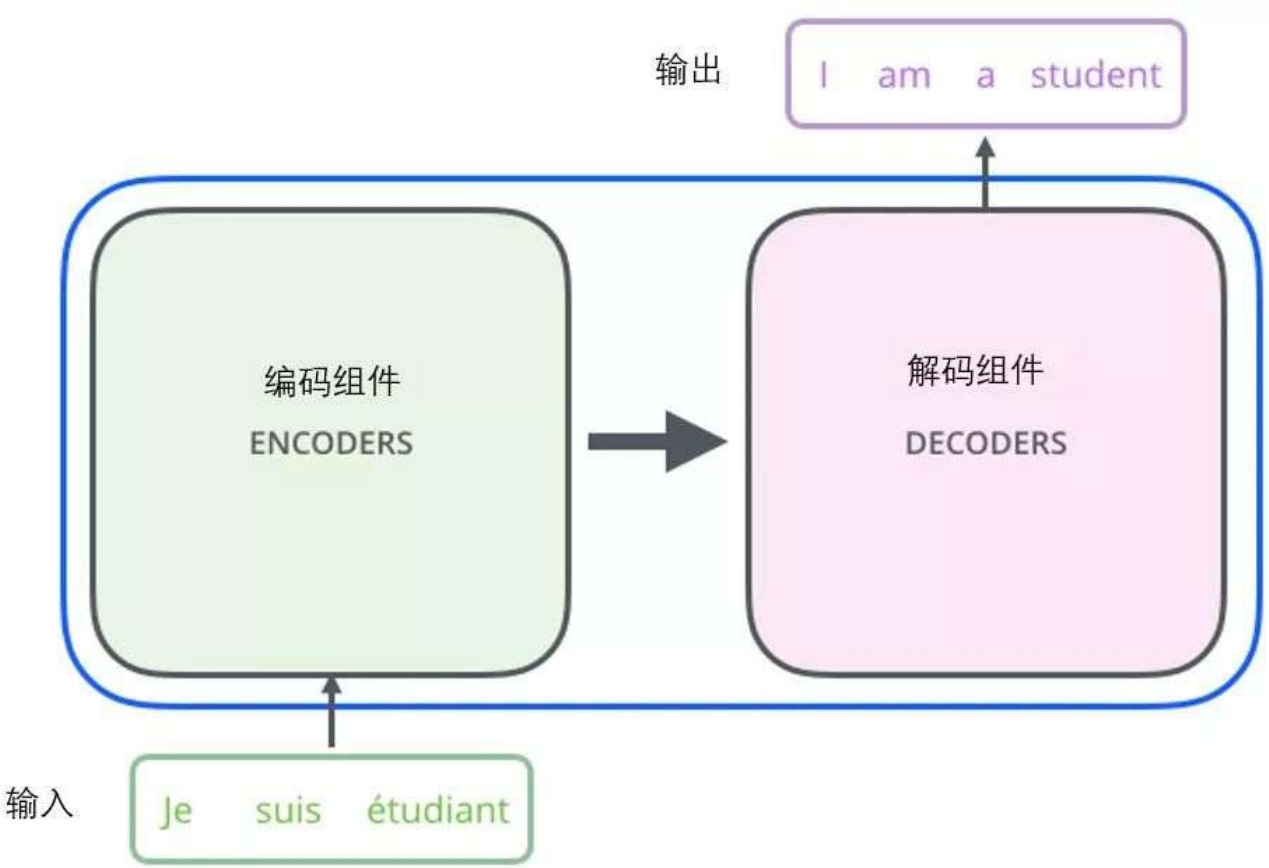
Transformer的工作流程

从宏观的视角开始 首先将这个模型看成是一个黑箱操作。在机器翻译中，就是输入一种语言，输出另一种语言。



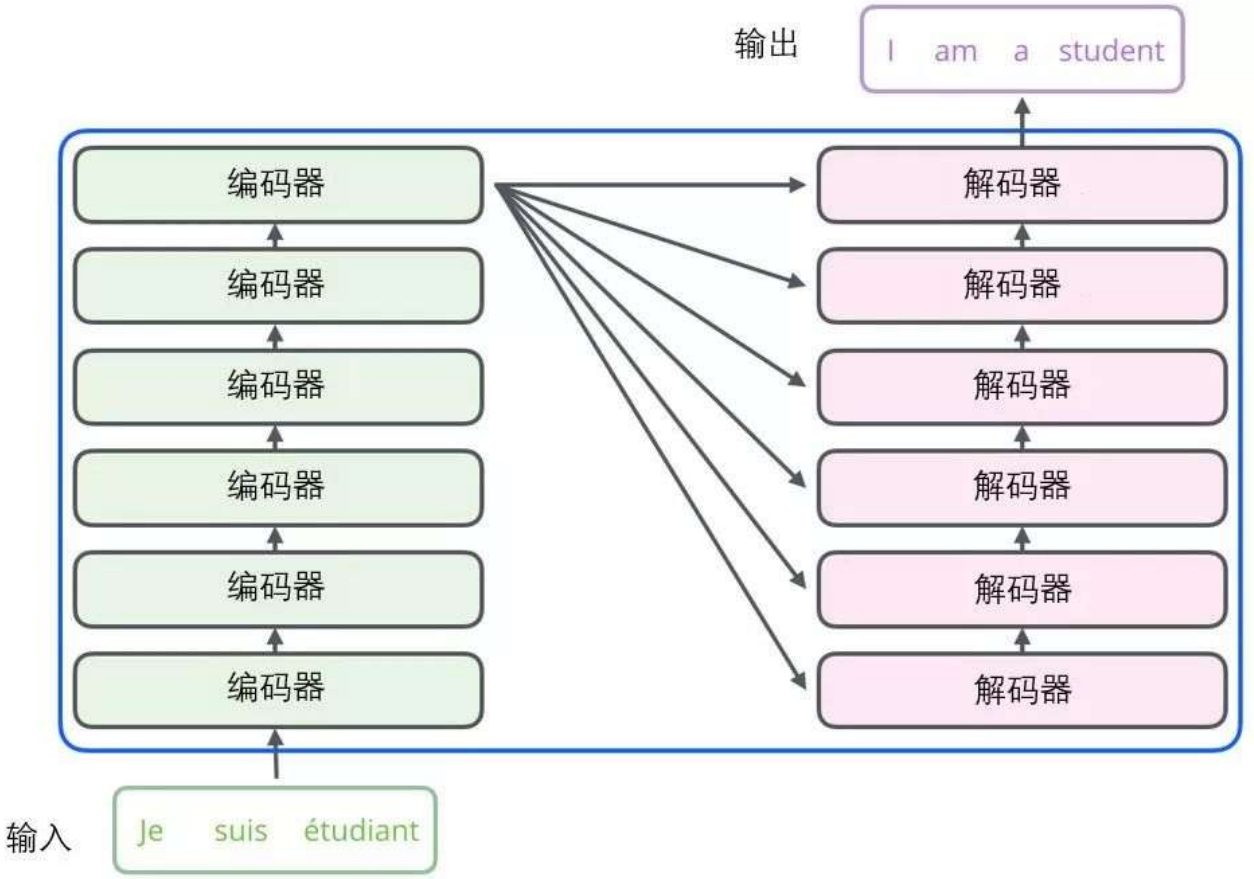
Transformer的工作流程

那么拆开这个黑箱，我们可以看到它是由编码组件、解码组件和它们之间的连接组成。



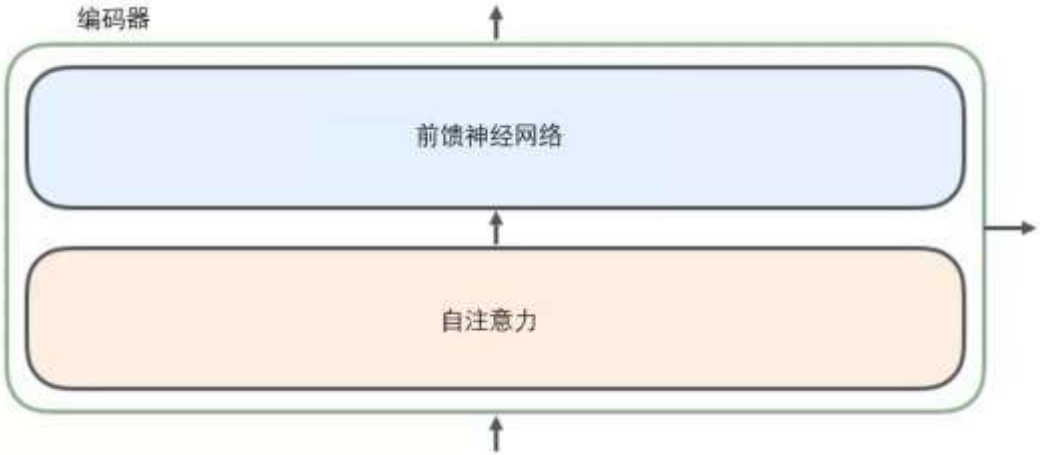
Transformer的工作流程

编码组件部分由一堆编码器（encoder）构成（论文中是将6个编码器叠在一起）。解码组件部分也是由相同数量（与编码器对应）的解码器（decoder）组成的。



Transformer的工作流程

所有的编码器在结构上都是相同的，但它们没有共享参数。每个解码器都可以分解成两个子层。

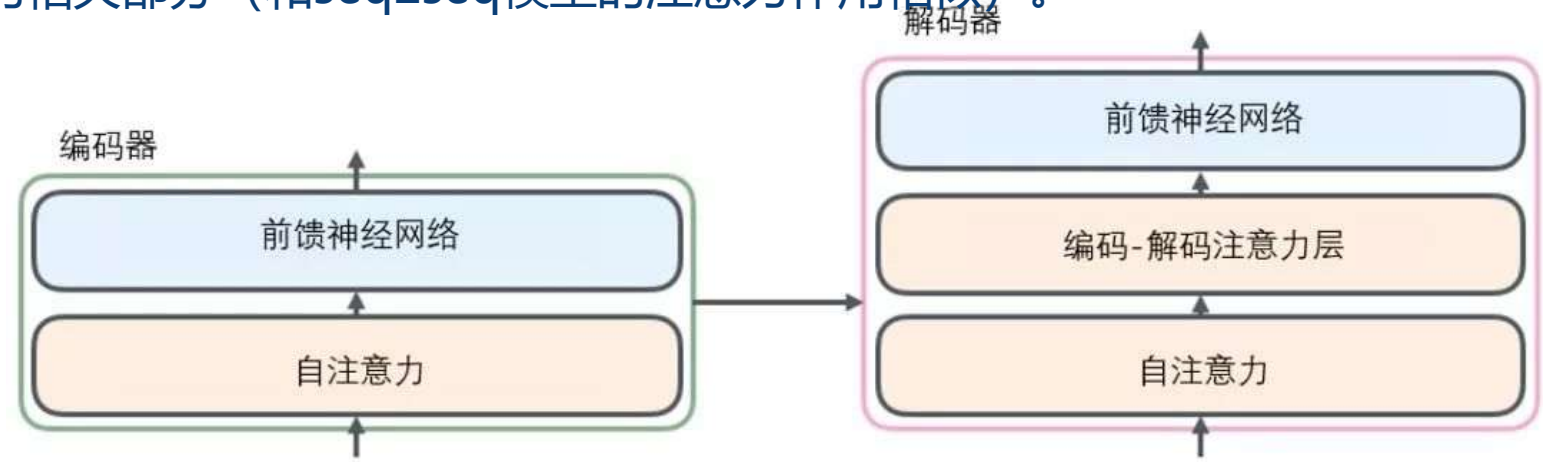


Transformer的工作流程

从编码器输入的句子首先会经过一个自注意力（self-attention）层，这层帮助编码器在对每个单词编码时关注输入句子的其他单词。

自注意力层的输出会传递到前馈（feed-forward）神经网络中。每个位置的单词对应的前馈神经网络都完全一样（译注：另一种解读就是一层窗口为一个单词的一维卷积神经网络）。

解码器中也有编码器的自注意力（self-attention）层和前馈（feed-forward）层。除此之外，这两个层之间还有一个注意力层，用来关注输入句子的相关部分（和seq2seq模型的注意力作用相似）。



Transformer的工作流程

各种向量或张量是怎样在模型的不同部分中，将输入转化为输出的。

像大部分NLP应用一样，我们首先将每个输入单词通过词嵌入算法转换为词向量。



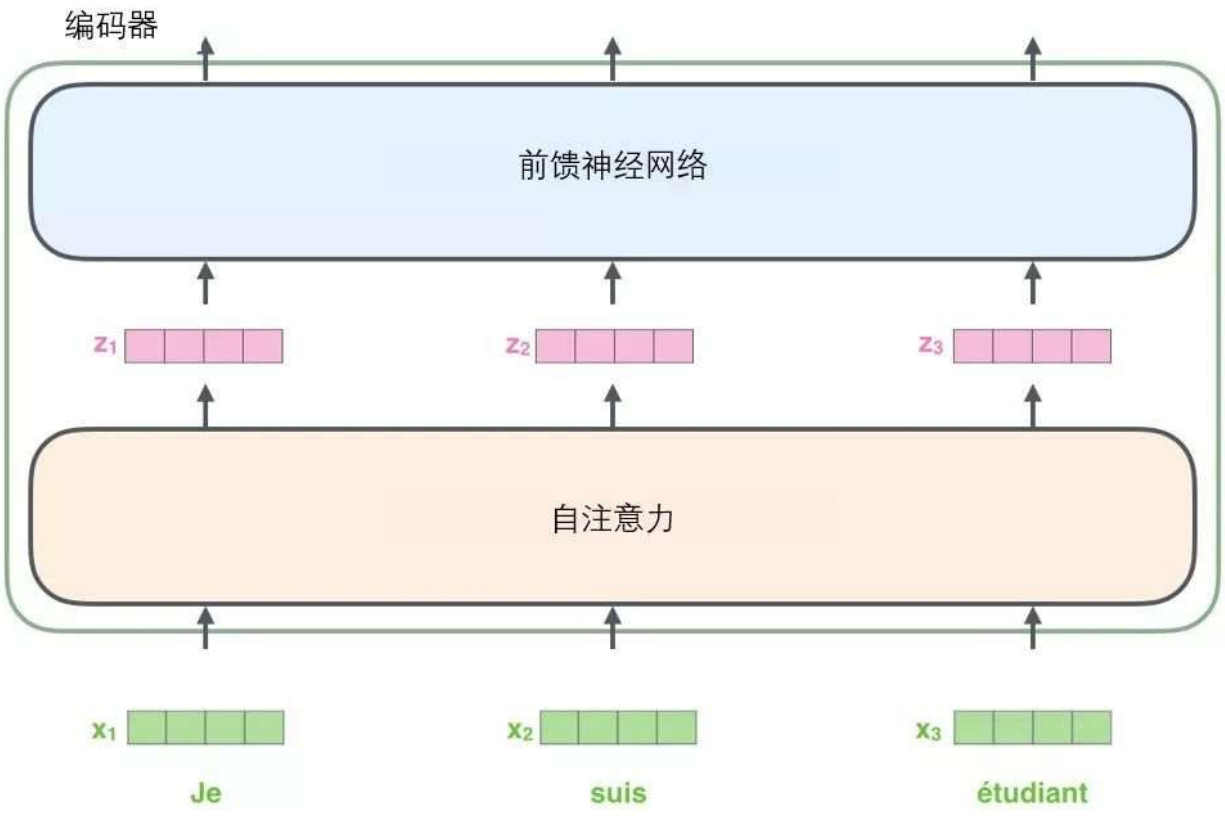
每个单词都被嵌入为512维的向量，词嵌入过程只发生在最底层的编码器中。所有的编码器都有一个相同的特点，即它们接收一个向量列表，列表中的每个向量大小为512维。在底层（最开始）编码器中它就是词向量，但是在其他编码器中，它就是下一层编码器的输出（也是一个向量列表）。

向量列表大小是我们可以设置的超参数:一般是我们训练集中最长句子的长度。

Transformer的工作流程

将输入序列进行词嵌入之后，每个单词都会流经编码器中的两个子层。

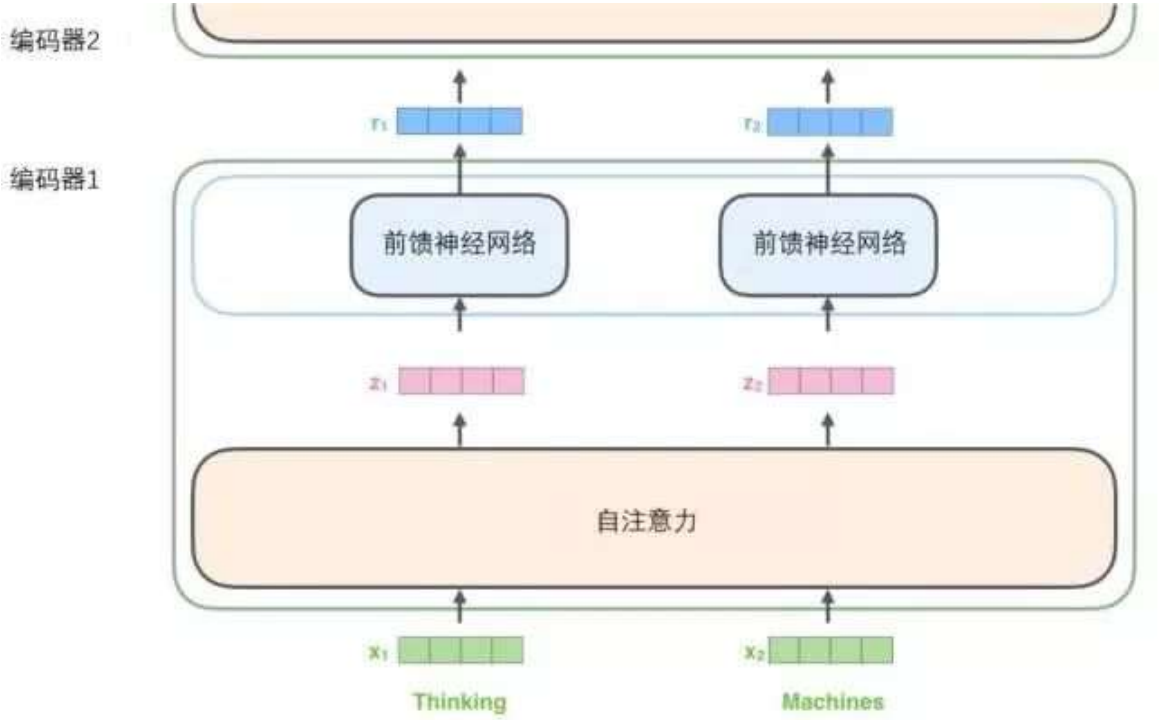
Transformer的一个核心特性，在这里输入序列中每个位置的单词都有自己独特的路径流入编码器。在自注意力层中，这些路径之间存在依赖关系。而前馈(feed-forward)层没有这些依赖关系。因此在前馈(feed-forward)层时可以并行执行各种路径。



Transformer的工作流程

编码过程

一个编码器接收向量列表作为输入，接着将向量列表中的向量传递到自注意力层进行处理，然后传递到前馈神经网络层中，将输出结果传递到下一个编码器中。



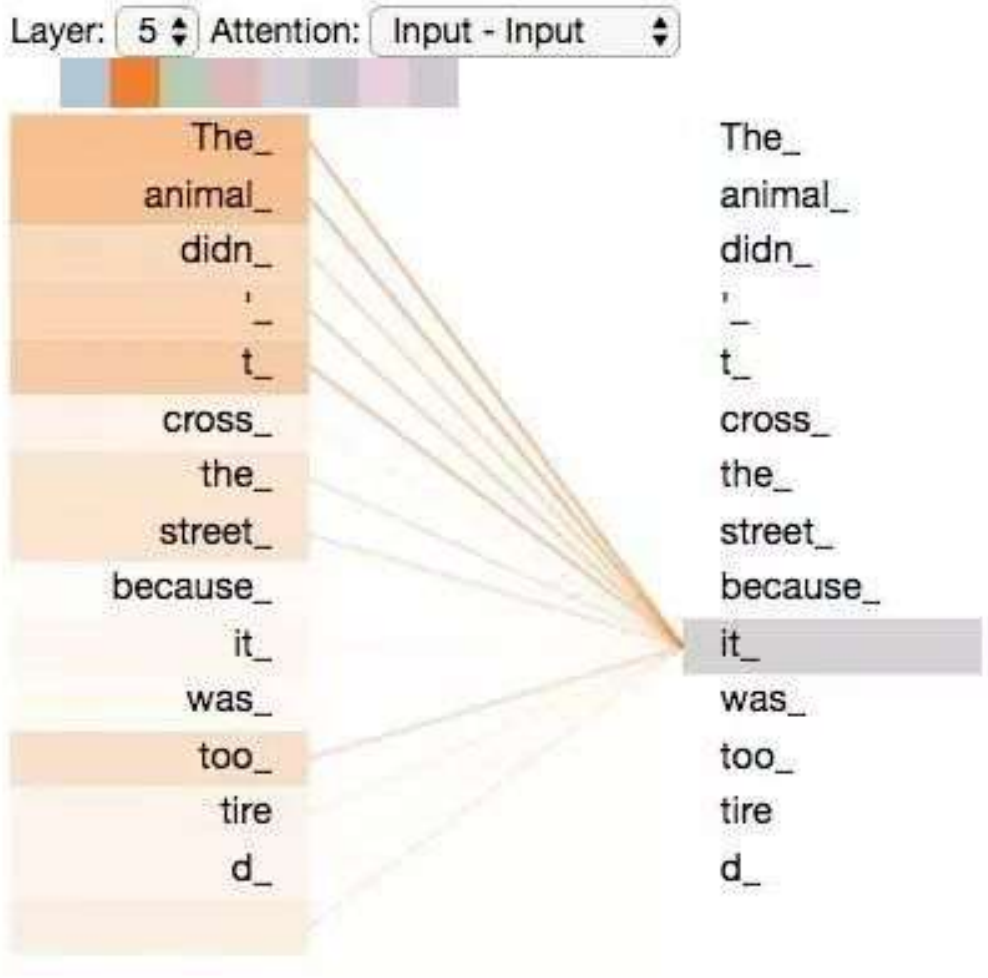
输入序列的每个单词都经过自编码过程。然后，它们各自通过前向传播神经网络：完全相同的网络，而每个向量都分别通过它。

Transformer的工作流程

从宏观视角看自注意力机制

随着模型处理输入序列的每个单词，自注意力会关注整个输入序列的所有单词，帮助模型对本单词更好地进行编码。

RNN会将它已经处理过的前面的所有单词/向量的表示与它正在处理的当前单词/向量结合起来。而自注意力机制会将所有相关单词的理解融入到我们正在处理的单词中。

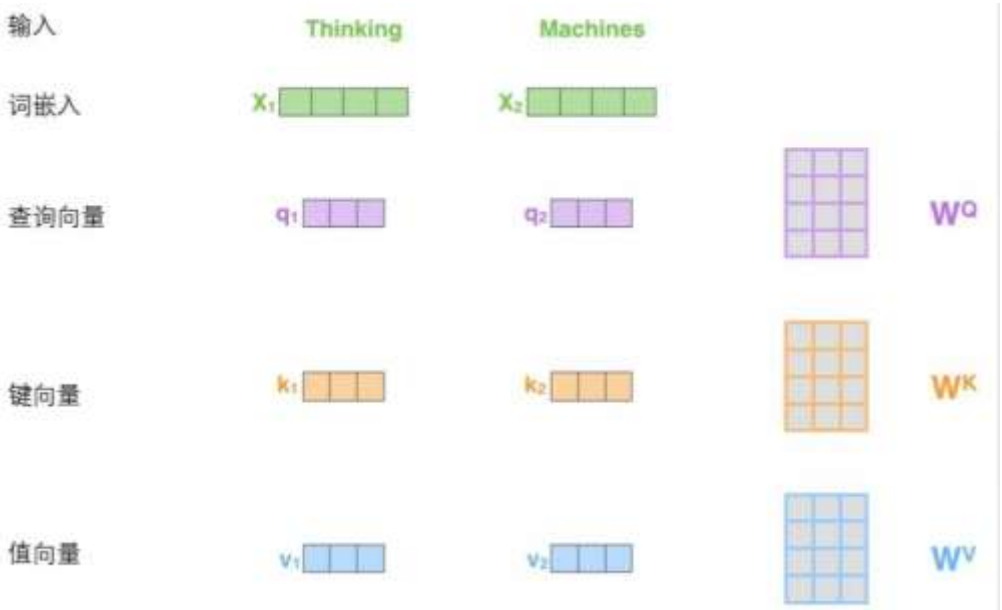


当我们在编码器#5（栈中最上层编码器）中编码“it”这个单词的时，注意力机制的部分会去关注“The Animal”，将它的表示的一部分编入“it”的编码中。

Transformer的工作流程

从微观视角看自注意力机制

计算自注意力的第一步就是从每个编码器的输入向量（每个单词的词向量）中生成三个向量。也就是说对于每个单词，我们创造一个查询向量(Q)、一个键向量(K)和一个值向量(V)。这三个向量是通过词嵌入与三个权重矩阵后相乘创建的，它们的维度是64，而词嵌入和编码器的输入/输出向量的维度是512. 但实际上不强求维度更小，这只是一种基于架构上的选择，它可以使多头注意力（multiheaded attention）的大部分计算保持不变。

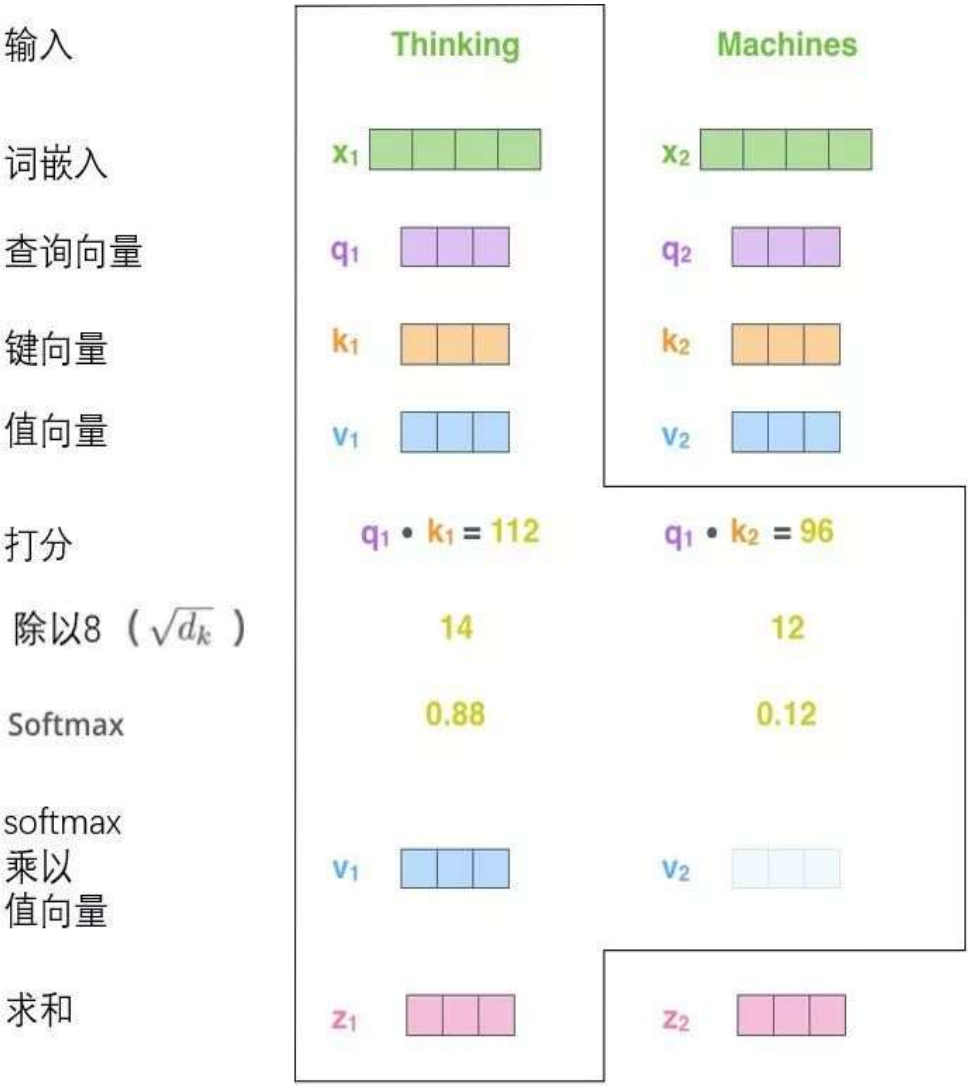


X_1与W_Q权重矩阵相乘得到q_1, 就是与这个单词相关的查询向量。最终使得输入序列的每个单词的创建一个查询向量Q、一个键向量K和一个值向量V。

Transformer的工作流程

什么是查询向量Q、键向量K和值向量V?

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



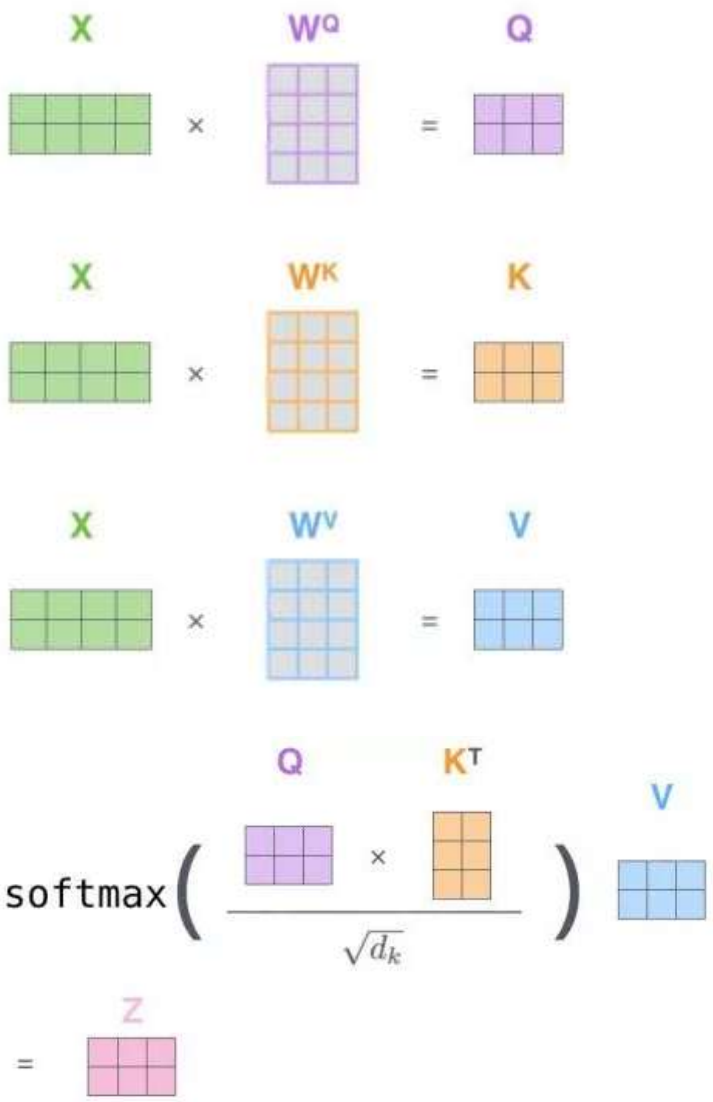
Transformer的工作流程

通过矩阵运算实现自注意力机制

第一步是计算查询矩阵、键矩阵和值矩阵。为此，我们将将输入句子的词嵌入装进矩阵X中，将其乘以我们训练的权重矩阵(WQ，WK，WV)。

x矩阵中的每一行对应于输入句子中的一个单词。我们再次看到词嵌入向量（512，或图中的4个格子)和q/k/v向量(64，或图中的3个格子)的大小差异。

最后，由于我们处理的是矩阵，我们可以用一个公式来计算自注意力层的输出。



Transformer的工作流程

“多头”注意力（“multi-headed” attention）的机制



一组Q,K,V得到了一组当前词的特征表达

Transformer的工作流程

“多头”注意力（“multi-headed” attention）的机制

1）将所有注意力头拼接起来



2）乘以矩阵W^0，它在模型中是联合训练的

X



3）结果是一个融合所有注意力头信息的矩阵Z，我们可以将其送到前馈神经网络

=



可以看到 Multi-Head Attention 输出的矩阵**Z**与其输入的矩阵**X**的维度是一样的。

Transformer的工作流程

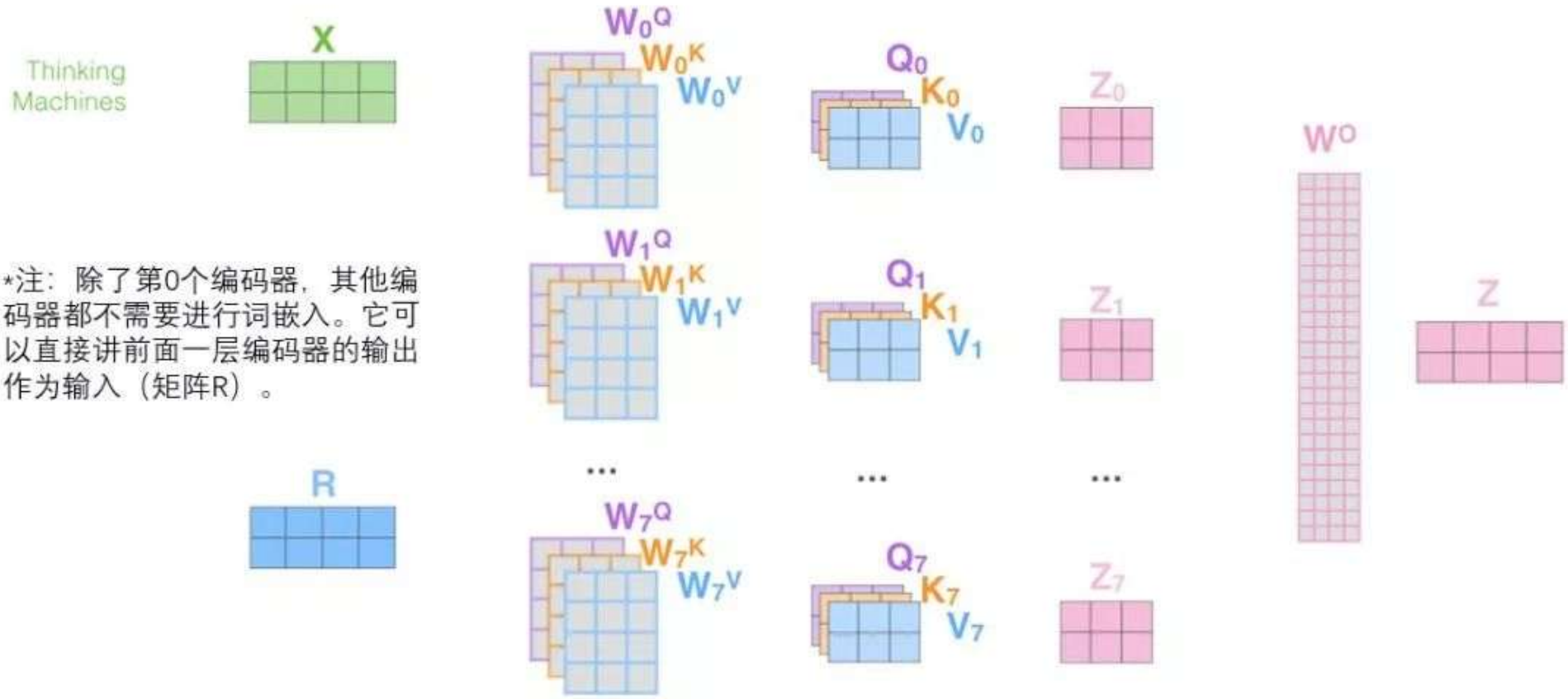
1）这是我们的输入句子*

2）编码每一个单词

3）将其分为8个头，将矩阵X或R乘以各个权重矩阵

4）通过输出的查询/键/值（Q/K/V）矩阵计算注意力

5）将所有注意力头拼接起来，乘以权重矩阵W^0

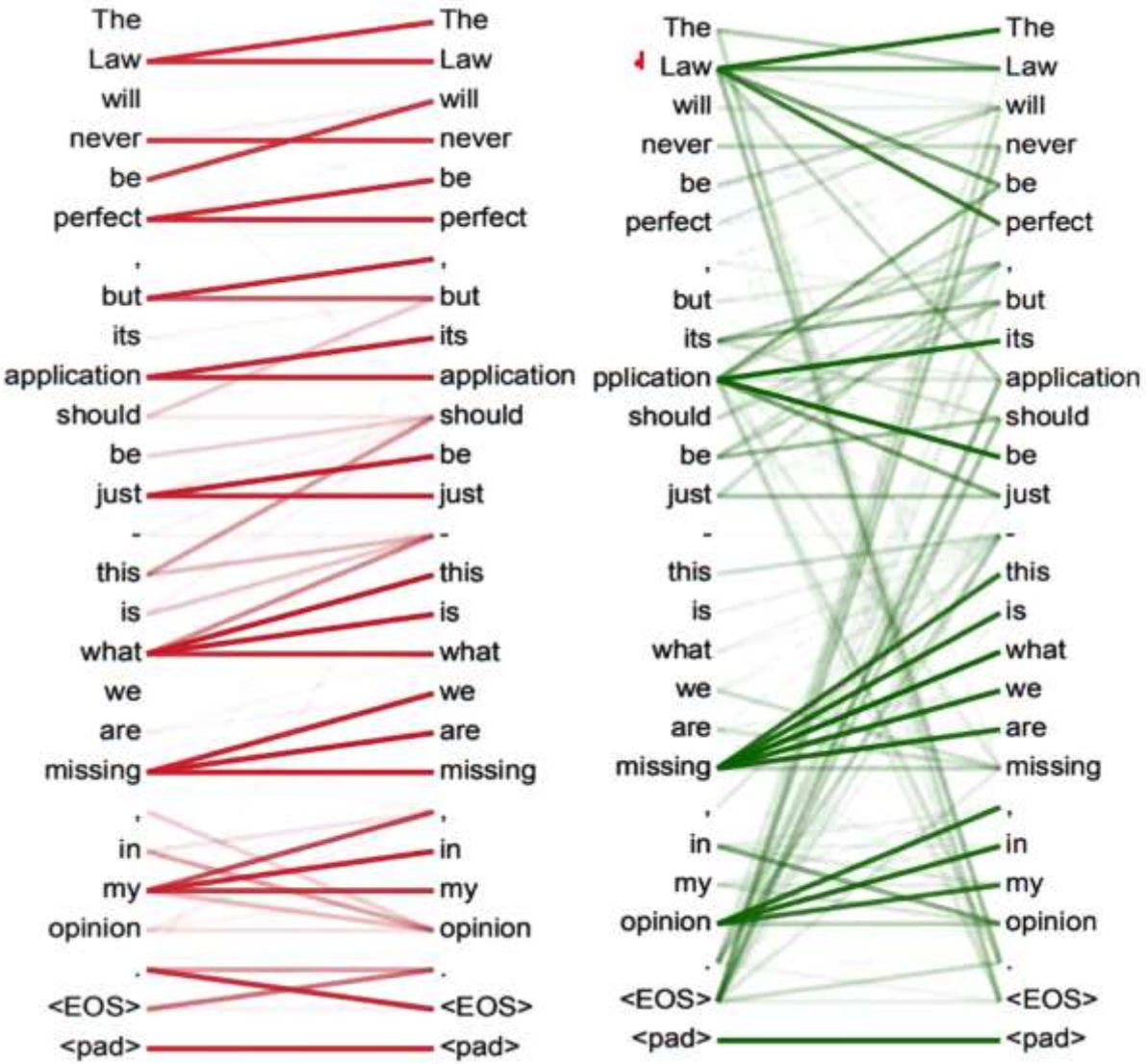


*注：除了第0个编码器，其他编码器都不需要进行词嵌入。它可以直接讲前面一层编码器的输出作为输入（矩阵R）。

Transformer的工作流程

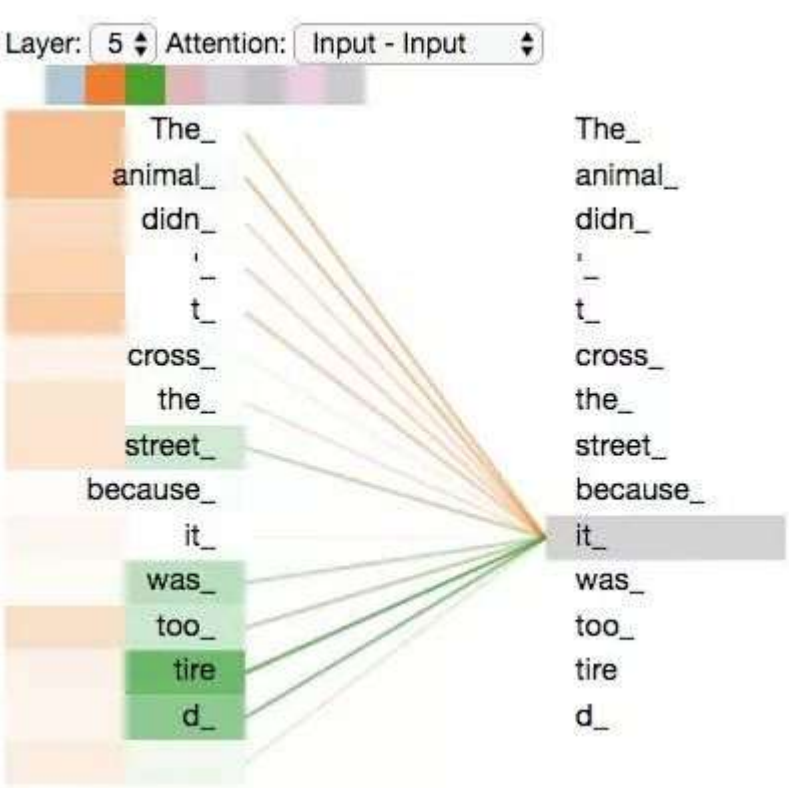
multi-headed结果

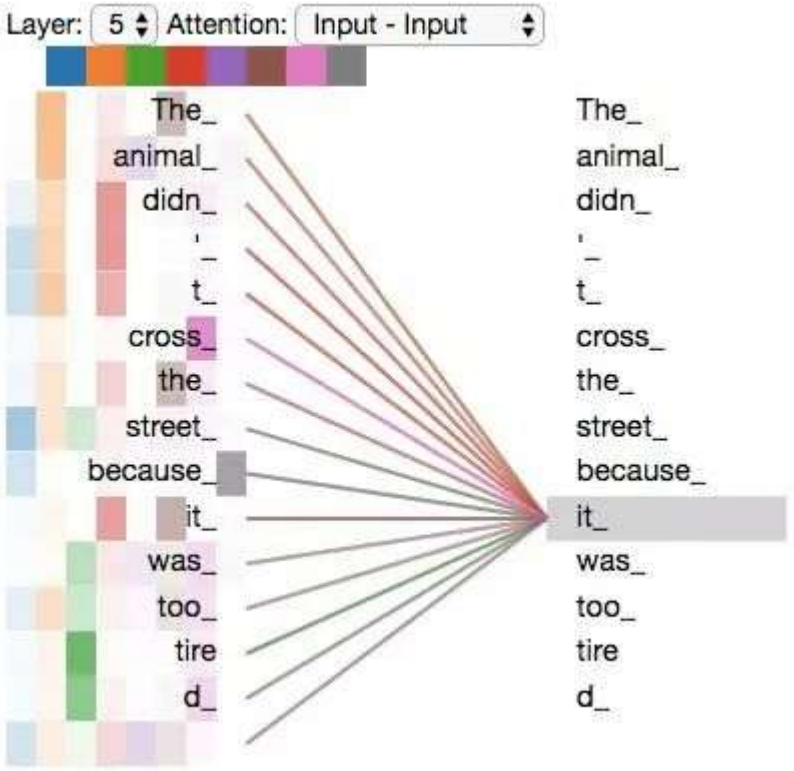
- 不同的注意力结果
- 得到的特征向量表达也不相同



Transformer的工作流程

当我们编码“it”一词时，一个注意力头集中在“animal”上，而另一个则集中在“tired”上，从某种意义上说，模型对“it”一词的表达在某种程度上是“animal”和“tired”的代表。
然而，如果我们把所有的attention都加到图示里，事情就更难解释了：



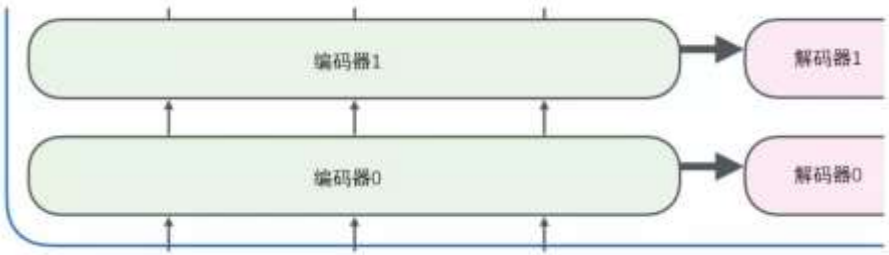


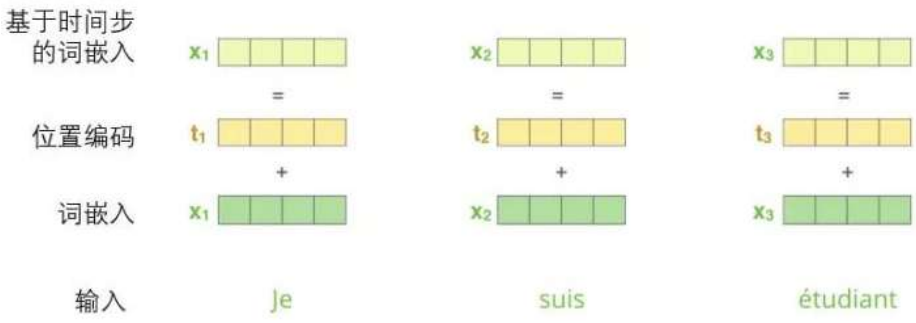
Transformer的工作流程

使用位置编码表示序列的顺序

到目前为止，我们对模型的描述缺少了一种理解输入单词顺序的方法。

为了解决这个问题，Transformer为每个输入的词嵌入添加了一个向量。这些向量遵循模型学习到的特定模式，这有助于确定每个单词的位置，或序列中不同单词之间的距离。这里的直觉是，将位置向量添加到词嵌入中使得它们在接下来的运算中，能够更好地表达的词与词之间的距离。





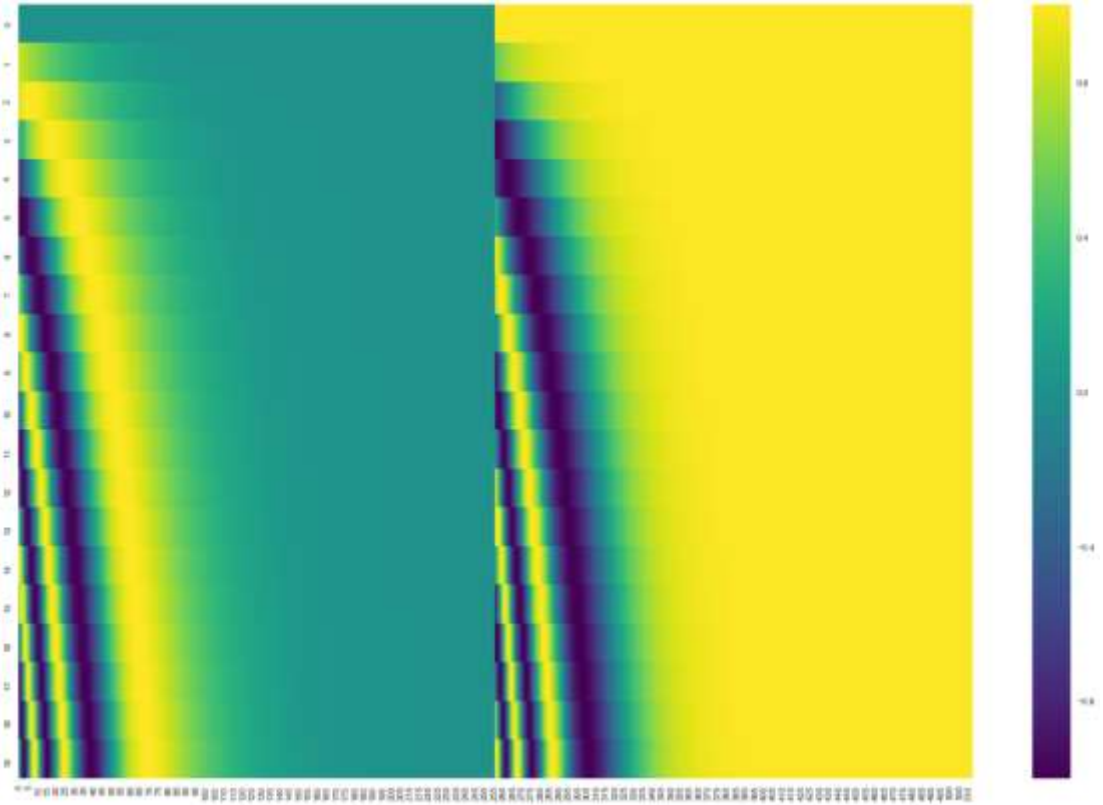
Transformer的工作流程

为了让模型理解单词的顺序，我们添加了位置编码向量，这些向量的值遵循特定的模式。
如果我们假设词嵌入的维数为4，则实际的位置编码如下：



Transformer的工作流程

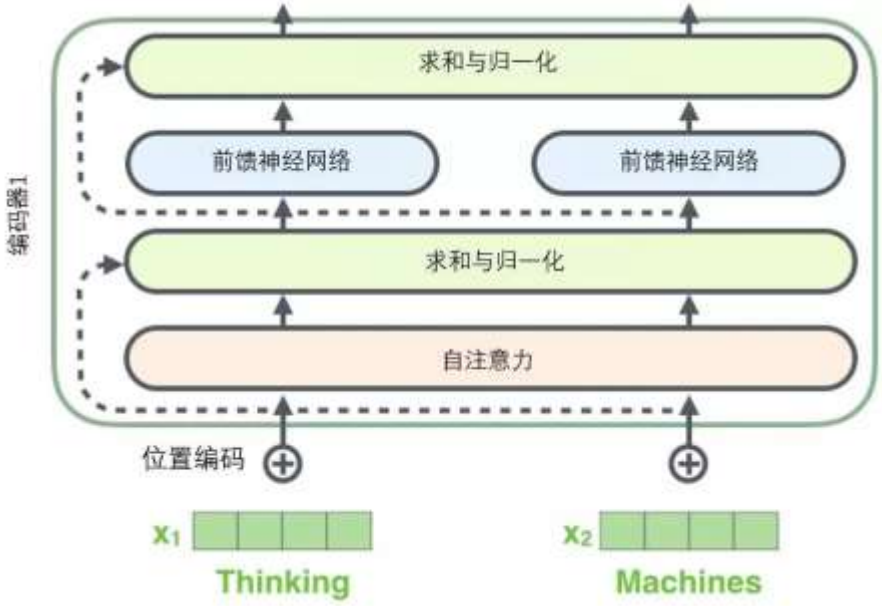
20字(行)的位置编码实例，词嵌入大小为512(列)。你可以看到它从中间分裂成两半。这是因为左半部分的值由一个函数(使用正弦)生成，而右半部分由另一个函数(使用余弦)生成。然后将它们拼在一起而得到每一个位置编码向量。



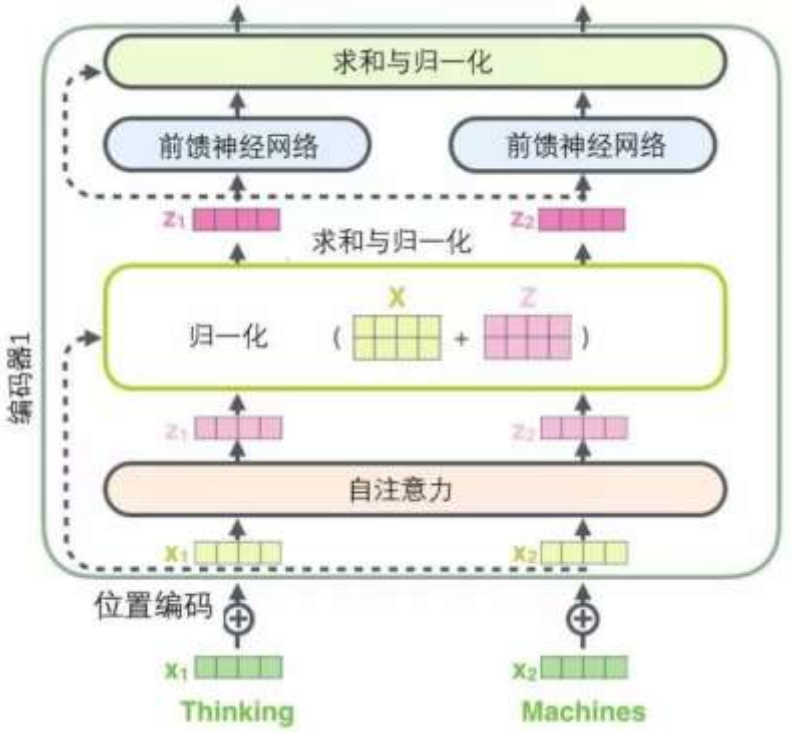
图中，每一行对应一个词向量的位置编码，所以第一行对应着输入序列的第一个词。每行包含512个值，每个值介于1和-1之间。我们已经对它们进行了颜色编码，所以图案是可见的。

Transformer的工作流程

在每个编码器中的每个子层（自注意力、前馈网络）的周围都有一个残差连接，并且都跟随着一个“层-归一化”步骤。

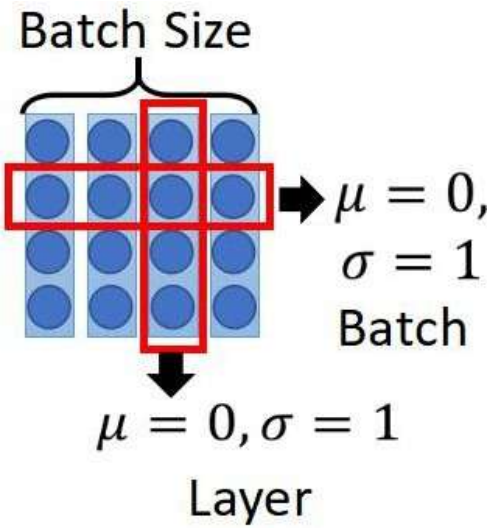


如果我们去可视化这些向量以及这个和自注意力相关联的层-归一化操作，那么看起来就像下面这张图描述一样：

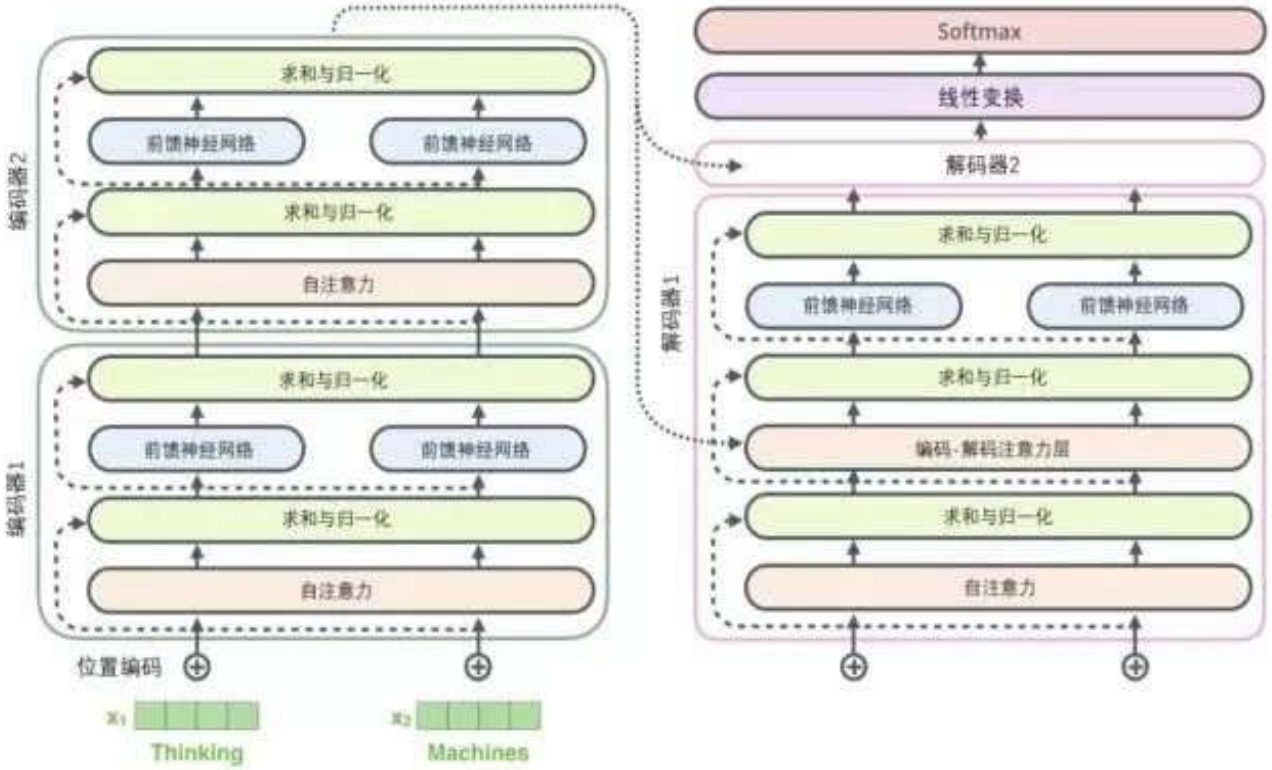


Transformer的工作流程

归一化：

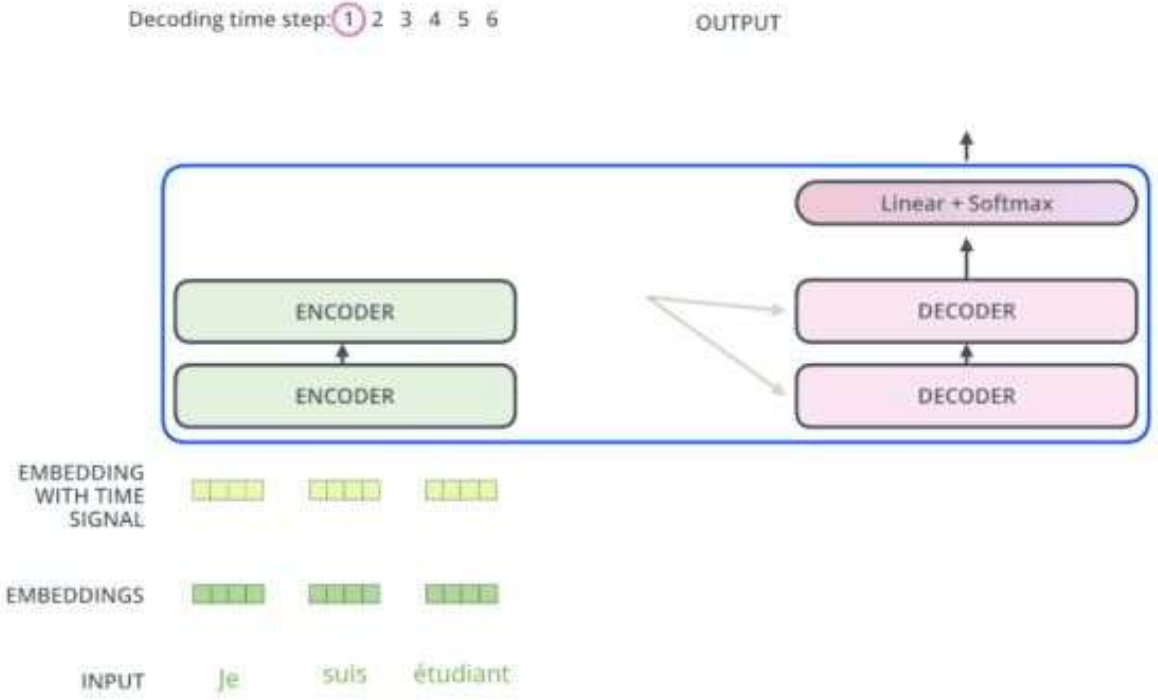


连接：基本的残差连接方式



Transformer的工作流程

编码器通过处理输入序列开启工作。顶端编码器的输出之后会变转化为一个包含向量K（键向量）和V（值向量）的注意力向量集 。这些向量将被每个解码器用于自身的“编码-解码注意力层”，而这些层可以帮助解码器关注输入序列哪些位置合适：



在完成编码阶段后，则开始解码阶段。解码阶段的每个步骤都会输出一个输出序列（在这个例子里，是英语翻译的句子）的元素

Transformer的工作流程

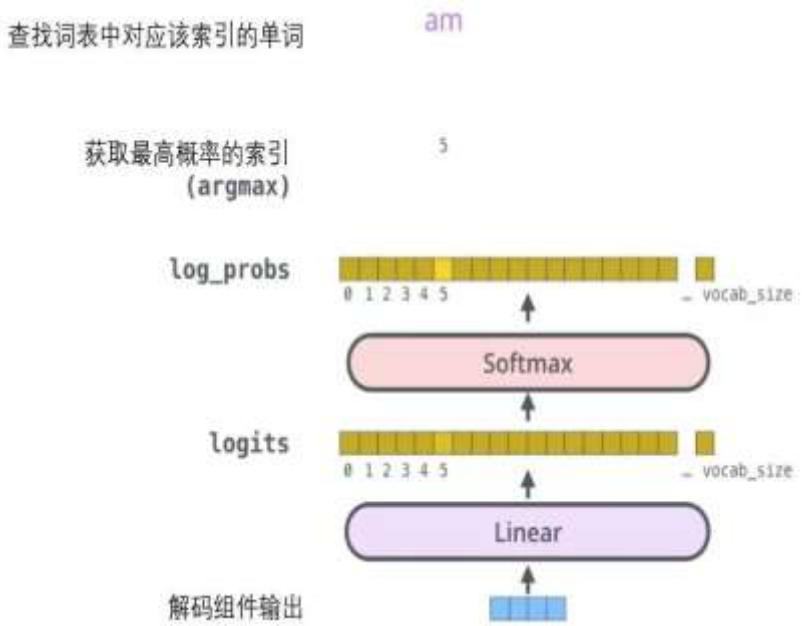
最终的线性变换和Softmax层

解码组件最后会输出一个实数向量。我们如何把浮点数变成一个单词？这便是线性变换层要做的工作，它之后就是Softmax层。

线性变换层是一个简单的全连接神经网络，它可以把解码组件产生的向量投射到一个比它大得多的、被称作对数几率（logits）的向量里。

不妨假设我们的模型从训练集中学习一万个不同的英语单词（我们模型的“输出词表”）。因此对数几率向量为一万个单元格长度的向量——每个单元格对应某一个单词的分数。

接下来的Softmax 层便会把那些分数变成概率（都为正数、上限1.0）。概率最高的单元格被选中，并且它对应的单词被作为这个时间步的输出。



这张图片从底部以解码器组件产生的输出向量开始。之后它会转化出一个输出单词。

本节参考

- Jay Alammar 博客文章《The Illustrated Transformer》 https://jalammar.github.io/illustrated-transformer/

大语言模型相关知识

预训练(Pre-Training)

为什么需要预训练(Pre-Training)？

- 预训练是为了让模型在见到特定任务数据之前，先通过学习大量通用数据来捕获广泛有用的特征，从而提升模型在目标任务上的表现和泛化能力。



数据准备

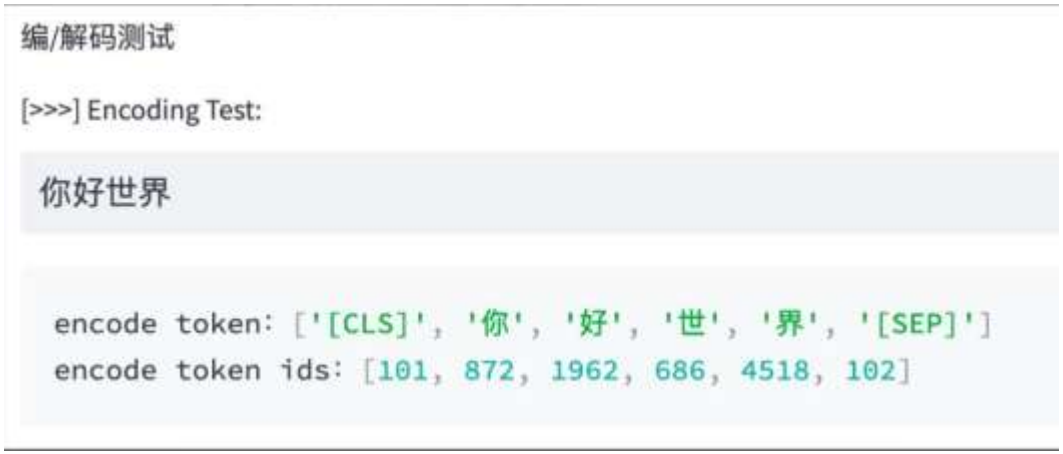
大语言模型的训练需要海量的优质数据作为支撑。这些数据就像是模型成长过程中的"营养餐"，其质量直接决定了模型的能力上限。数据准备的过程极其复杂和重要，需要多个环节的精心处理。

- **数据的收集和清洗**：

  去除垃圾信息，确保语言质量，处理隐私信息，以及过滤有害内容

- **数据标准化和编码**：

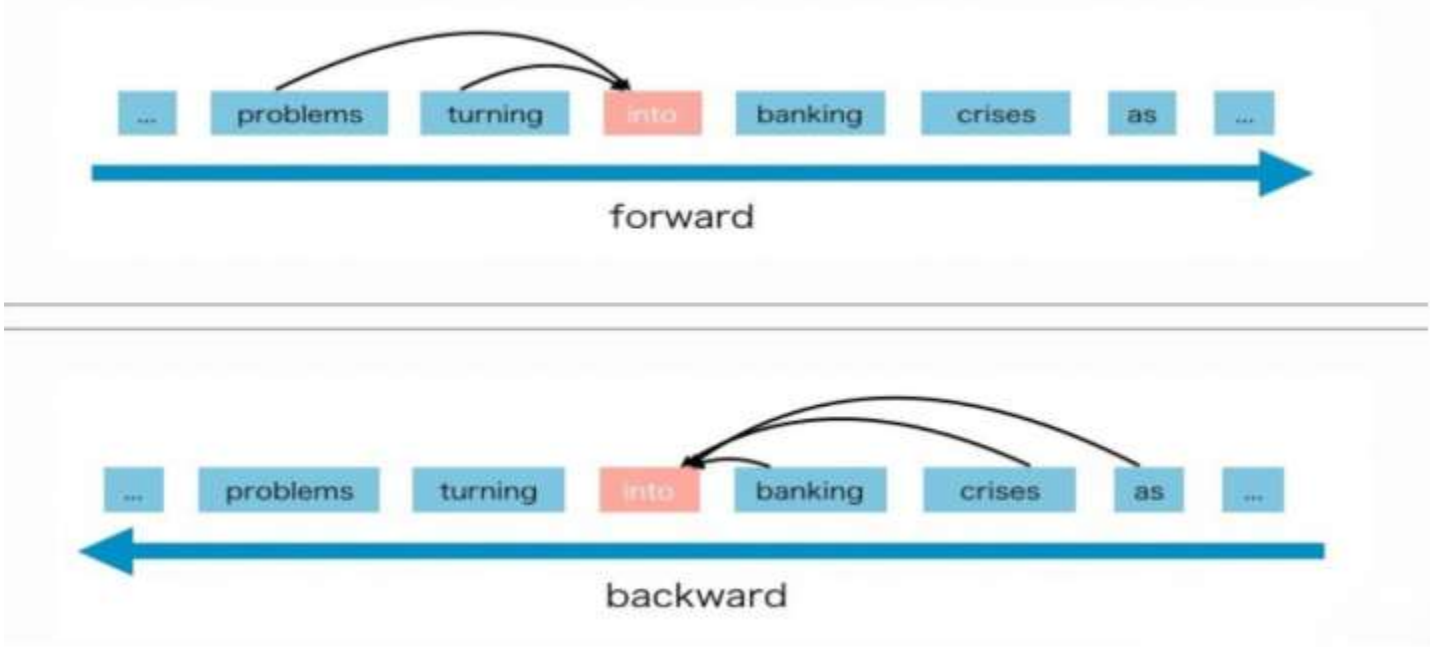
  将原始的文本数据需要转换成模型能够理解的数字格式，其中最重要的是分词操作（Tokenization）和嵌入操作（Embedding）



自回归(Autoregressive)模型

自回归(Autoregressive)语言模型

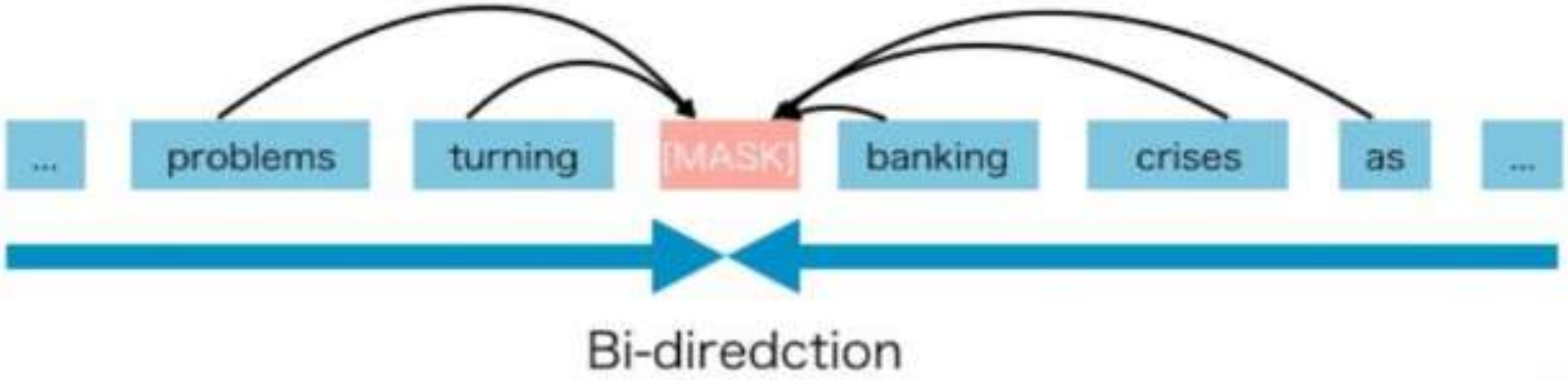
- 这种方法主要用于生成式预训练（如GPT）。模型通过预测文本序列中的下一个token来学习语言规律。即在自回归模型中，输入序列中的每个token都是基于前文信息进行预测的，也就是根据前面所有的词预测下一个词。



自编码(Autoencoder)模型

自编码(Autoencoder)语言模型

- 这种方法主要用于BERT类的模型，它通过预测文本中被遮蔽（masked）的token来进行训练。即输入序列中的一部分token被“遮蔽”掉（通常使用一个特殊的[MASK]标记代替），模型的目标是根据上下文预测出这些被遮蔽的token。



微调(Fine-Tuning)

为什么需要微调(Fine-Tuning)？

- 尽管预训练模型已经在大规模数据集上学到了丰富的通用特征和先验知识，但这些特征和知识可能并不完全适用于特定的目标任务。 微调通过在新任务的少量标注数据上进一步训练预训练模型，使模型能够学习到与目标任务相关的特定特征和规律，从而更好地适应新任务。

新任务数据集上微调

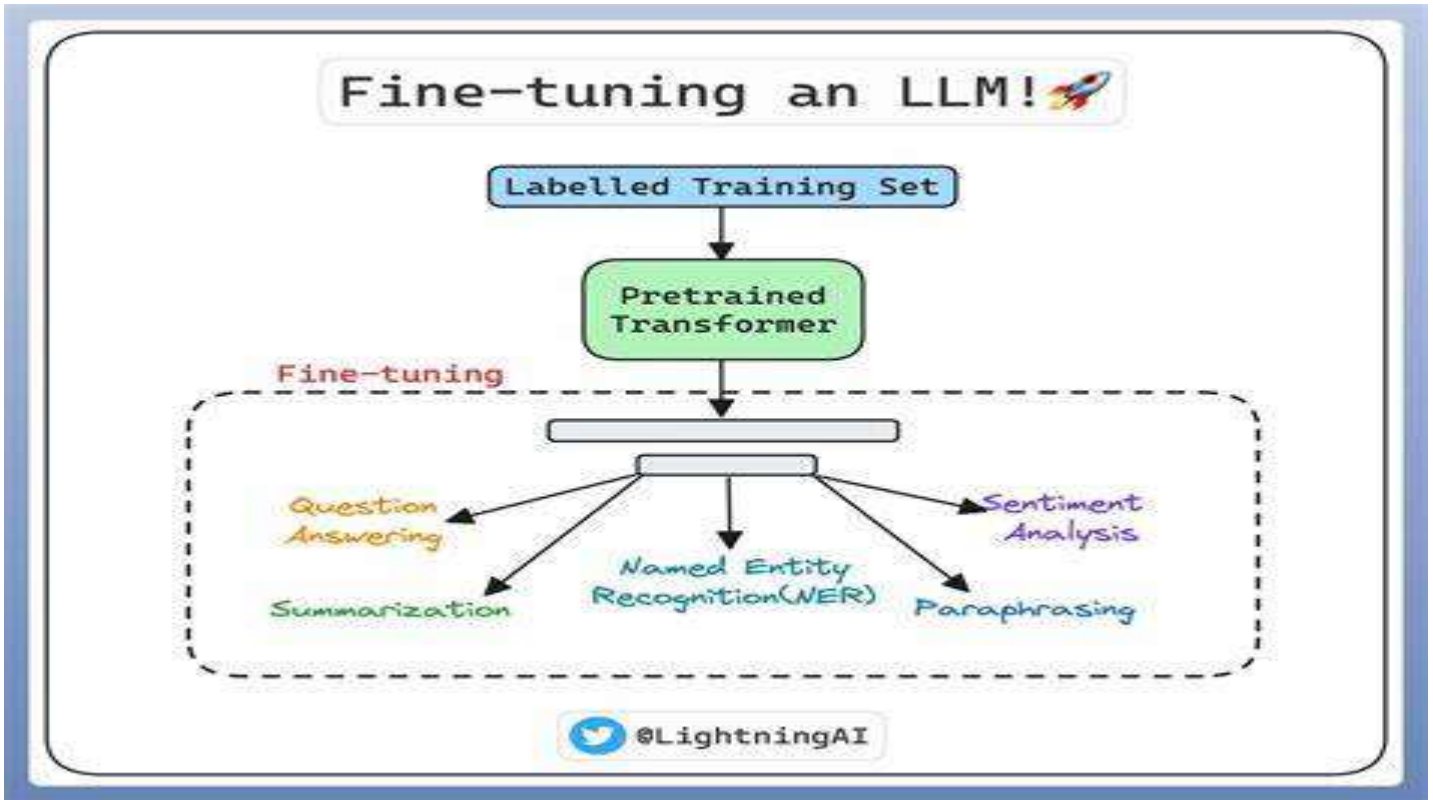
- **监督微调SFT**
  - 使用标注过的带有标签数据来调整预训练模型的参数，使其更好的适应特定任务或领域。

- **基于人类反馈的强化学习微调RLHF**
  - 一种利用人类反馈作为奖励信号来训练强化学习模型的方法，旨在提升模型生成文本等内容的质量，使其更符合人类偏好。

全量微调

口 全量微调(Full Fine-Tuning)

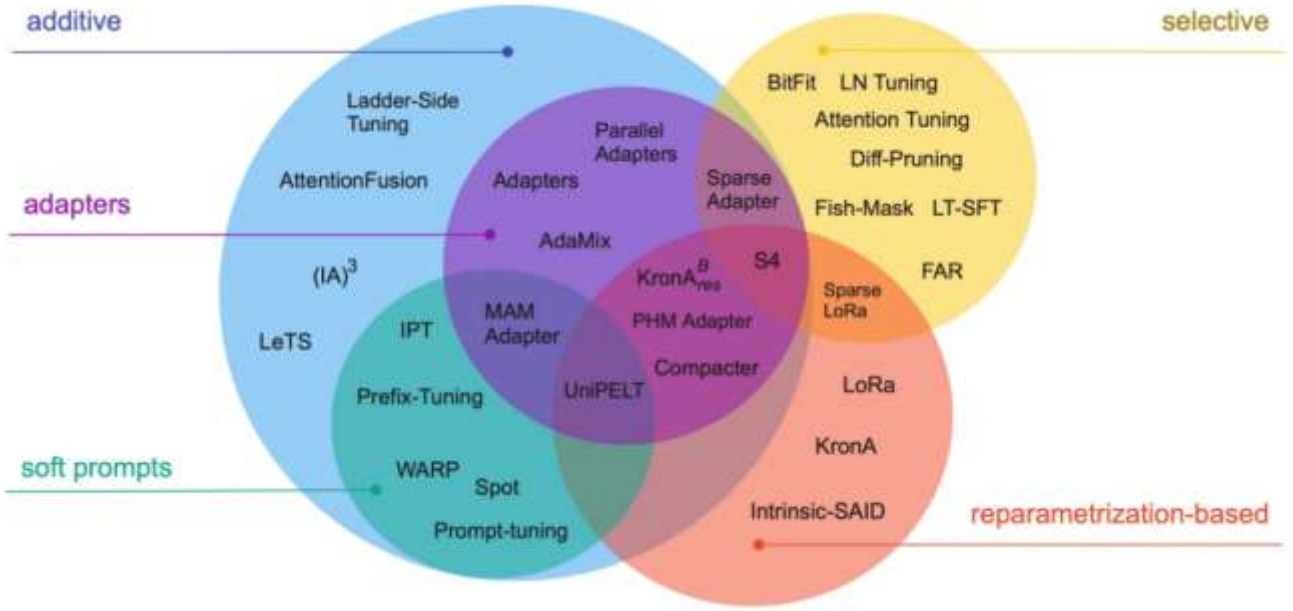
- 全量微调利用特定任务数据调整预训练模型的所有参数，以充分适应新任务。它依赖大规模计算资源，但能有效利用预训练模型的通用特征。



参数高效微调

参数高效微调PEFT

- PEFT旨在通过最小化微调参数数量和计算复杂度，实现高效的迁移学习。它仅更新模型中的部分参数，显著降低训练时间和成本，适用于计算资源有限的情况。比如：LoRA，Adapters等



提示工程(Prompt Engineering)

□ Prompt是什么?

- Prompt是指一段用于引导模型理解问题的文本，通常位于问题的开头或结尾。它用于为模型提供上下文和任务要求，以帮助模型更好地理解和回答问题。

例如：想让ChatGPT题目是“我有一个朋友叫张三”的一篇文章，那么prompt 就可以写成：

请以“我有一个朋友叫张三”为开头，写一篇关于张三的文章，不少于500字。

我有一个朋友叫张三。他是一个值得我深入了解和写下来分享的人。张三是一个勤奋而才华横溢的个体，他对生活充满热情，对追求卓越不遗余力。首先，张三在学术上有着非凡的成就。他是一名优秀的全才。在学校里，他总是在各个学科中表现出色，从数学到科学，从文学到艺术，他总能轻松应对。他对知识的渴求使他成为了学校里所有老师们的骄傲。除了学习上的成功，张三还是一个多才多艺的人。他擅长音乐，会弹奏多种乐器，无论是言他还是铜琴，他都能轻松驾驭。他的演奏技巧和窗乐感受力让人叹为观止。不仅如此，他还擅长给画和写作，他的画作和文字总是充满着想象力和创造力。他在各种艺术领域中的才华使他成为了众人眼中的瑰宝。然而，张三的成功并非仅仅来自于他的天赋。他努力工作，从不轻富放弃。无论是在学术上，还是在艺术创作中，他总是付出百分之百的努力。他不断探索新的领域，勇敢地画对挑战。饱的毅力和决心是他成功的关键要素。更重要的是，张三是一个热爱助人的人。他经常参与各种慈善活动，帮助那些需要帮助的人。他无私奉献自己的时间和精力，为那些身处困境中的人们提供帮助和支持。他的善良和友善使他成为了身边朋友们的榜样。我很庆幸能有张三这样的朋友。他的成功和品质对我有着深远的影响。他不仅鼓舞着我继续追求自己的梦想，还教会了我如何通过努力和毅力来实现目标。他是一个华丽的典范。总之，我的朋友张三以他的才华、努力和善良赢得了人们的尊重和欣赏。他的成功故事给我们每个人都带来了启示。张三是我心目中的英雄，他的故事一定会激励更多人去追求卓越和成就伟大!

这是一种**最简单、随意**的Prompt写法，对于简单任务也能产生可以接受的结果。然而，如果想要利用大模型对复杂的下游任务进行赋能，prompt的写法对于效果的保证就显得至关重要了。

Zero-shot Prompt

Zero-shot顾名思义就是零样本，就是在prompt中**不加任何和任务相关的示例。**

Zero-shot prompt

Prompt:
将文本分为中性、负面或积极
文本：我觉得这次假期还不错

Output:中性

在上述提示中，并没有提供任何带有分类的文本示例给模型，大型语言模型已经理解了“情感”,这就是其zero-shot的体现。

Few-shot Prompt

Few-shot Prompt是指在prompt中**加入和推理任务相同的推理样例。**

如任务：使用一个词语进行造句。

Few shot prompt

Prompt:

“whatpu”是一种生长在坦桑尼亚的小型毛茸动物。

使用“whatpu”一词的句子示例是：我们在非洲旅行时看到了这些非常可爱的whatpu。

“farduddule”是指快速地上下跳跃。

使用“farduddule”一词的句子示例是：

Output**当我们赢得比赛时，我们都开始farduddle来庆祝**

- 对于更困难的任务，可以**尝试增加示例**（例如3-shot、5-shot、10-shot等）进行实验。

Few-shot Prompt

Few-shot prompt的限制：在处理复杂任务（如推理任务）时，Few-shot的效果可能会不好。比如：判断一个数字序列之和是否是偶数。

Few-shot prompt

Prompt:

这组中的奇数加起来是一个偶数：4、8、9、15、12、2、1。
解答：答案是假的。

这组中的奇数加起来是一个偶数：17、10、19、4、8、12、24。
解答：答案是真的。

这组中的奇数加起来是一个偶数：16、11、14、4、8、13、24。
解答：答案是真的。

这组中的奇数加起来是一个偶数：17、9、10、12、13、4、2。
解答：答案是假的。

这组中的奇数加起来是一个偶数：15、32、5、13、82、7、1 。
解答：

Output: 答案是真的。

- 对于这种类型的推理问题，Few-shot prompt不足以获得可靠的回答。
- 总体而言，提供示例在解决某些任务时是有用的。当零样本提示和少样本提示不足时，这可能意味着**模型所学到的内容不足以在任务上表现良好。**

思维链(CoT)

思维链是什么?

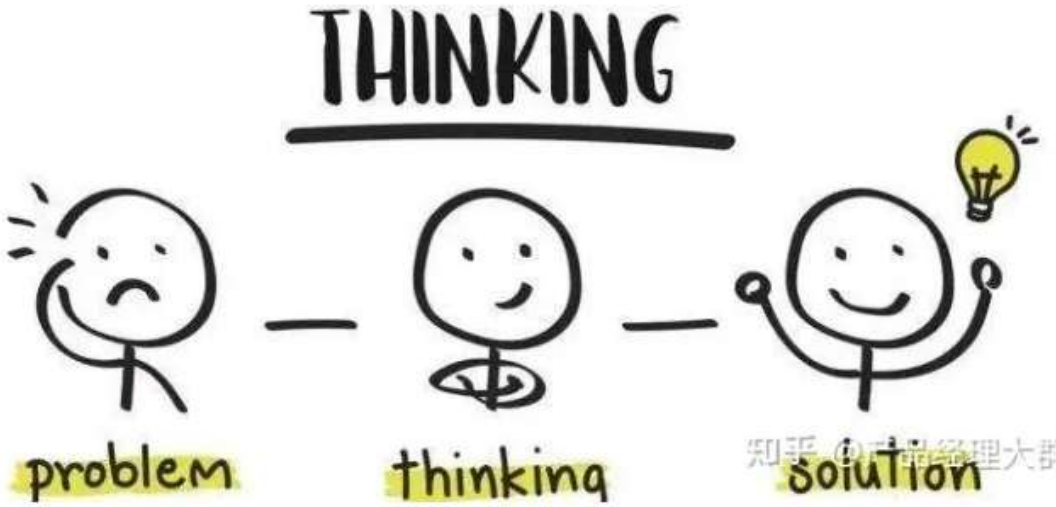
- 通过让大模型逐步参与将一个复杂问题分解为一步一步的子问题并依次进行求解的过程可以显著提升大模型的性能。而这些推理的中间步骤就被称为思维链(Chain of Thought)



思维链(CoT)

思维链的优势

- 推理能力
- 可解释性
- 可控性
- 灵活性



Zero-shot CoT

Chain of thoughts（CoT）技术通过引导大模型先给出推理过程，再给出最终答案。CoT引导方法分为两种：zero-shot CoT和few-shot CoT。

zero-shot CoT在zero-shot的基础上添加指令“**一步一步的思考给出答案**”。如下图：

Zero-shot prompt

Prompt:

一个杂耍者可以同时抛16个球。这些球的一半是高尔夫球，而高尔夫球的一半是蓝色的。那么有多少个蓝色的高尔夫球呢？A:答案是

Output: 8

Zero-shot CoT prompt

Prompt:

一个杂耍者可以同时抛16个球。这些球的一半是高尔夫球，而高尔夫球的一半是蓝色的。那么有多少个蓝色的高尔夫球呢？请一步一步思考后给出答案。

Output: 首先，我们知道一半的球是高尔夫球，即16球的一半，也就是8个高尔夫球。其次，高尔夫球的一半是蓝颜色的，所以需要计算8个高尔夫球的一半，即8球的一半，也就是4个蓝色的高尔夫球。所以，答案是4个蓝色的高尔夫球。

Few-shot CoT

Few-shot

该组中的奇数加起来为偶数：4、8、9、15、12、2、1
答：答案是假的。

该组中的奇数加起来为偶数：17、9、10、12、13、4、2
答：答案是假的。

该组中的奇数加起来为偶数：15、32、5、13、82、7、1
答：

答案是**真的**。 ❌

Few-shot CoT

该组中的奇数加起来为偶数：4、8、9、15、12、2、1。
答：将所有奇数（9、15、1）相加得到 25。答案是假的。

该组中的奇数加起来为偶数：16、11、14、4、8、13、24。
答：将所有奇数（11,13）相加得到24。答案是真的。

该组中的奇数加起来为偶数：17、9、10、12、13、4、2。
答：将所有奇数（17、9、13）相加得到 39。答案是假的。

该组中的奇数加起来为偶数：15、32、5、13、82、7、1。
答：

将所有奇数（15、5、13、7、1）相加得到 41，答案是**假的**。 ✅

自一致性(Self-consistency)

- **最优的答案出现的次数最多**

问：当我六岁的时候，我姐姐的年龄只有我的一半。现在我70岁了，我姐姐几岁?
答：35

问：树林里有 15 棵树。树林里的工人今天将在树林里植树。他们完成后， 将有21棵树。今天果园工人种了多少棵树?
答：我们从 15 棵树开始。后来我们就有了 21 棵树。差别一定是他们种植的树的数量。 所以，他们一定种了 21 - 15 = 6 棵树。答案是6。
问：肖恩有五个玩具。圣诞节时，他从爸爸妈妈那里各收到了两个玩具。他现在有多少个玩具?
答：他有5个玩具。他从妈妈那里得到了 2 个，所以之后他就有了 5 + 2 = 7 个玩具。然后他又从爸爸那里得到了 2 个，所以他总共有 7 + 2 = 9 个玩具。答案是9。
问：服务器机房有九台电脑。从周一至周四每天另外安装五台计算机。服务器机房现在有多少台计算机?
答：周一到周四有4天。每天添加 5 台计算机。这意味着总共 4 * 5 = 添加了 20 台计算机。一开始有 9 台计算机，所以现在有 9 + 20 = 29 台计算机。 答案是29。
……
问：当我 6 岁的时候，我姐姐的年龄只有我的一半。现在我70岁了，我妹妹几岁了?

答1：我6岁的时候，我姐姐的年龄只有我的一半，所以她才3岁。现在我70岁了，所以她是70 - 3 = 67。**答案是67。**
答2：叙述者6岁时，他的妹妹只有他的一半大，也就是3岁。现在叙述者是 70 岁，他的妹妹将是 70 - 3 = 67 岁。**答案是67。** 答3：我6岁的时候，我姐姐的年龄只有我的一半，所以她才3岁。现在我70岁了，所以她是70/2 = 35。**答案是35。**

自一致性(Self-consistency)



给GPT-4V这张图片，要求它计算图中船只数量

根据**自洽**，推断出图中**有4条船。**

Prompt: 计算图片中船只的数量，描述它们的每个位置。
GPT-4V: 图中**共有4艘船。** 1）第一艘船在图片左侧，停靠在码头。这是一艘较大的船，顶部有一座塔。 2）第二艘船位于图像中央，也停靠在码头。这是一艘较小的船，顶部有一个白色的天篷。 3）第三艘船在图片右侧，停靠在码头。这是一艘较小的船，顶部有一个白色的天篷，与第二艘船类似。 4) 第四艘船位于背景中，位于图像的最右侧。那是一艘帆船，看上去正在水面上航行。

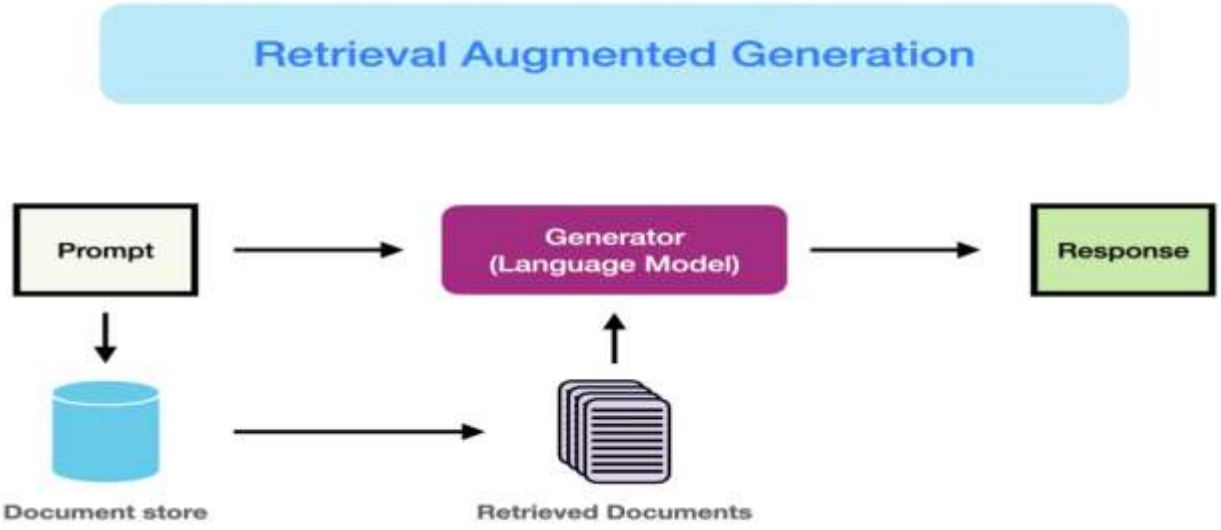
Prompt: 计算图片中船只的数量。
GPT-4V: 图像中**有6艘船。**

Prompt:计算图像中的船只数量。描述它们各自的位置。
GPT-4V: 图中**有 4 艘船。** 1）第一艘船在图片左侧，停靠在码头。这是一艘较大的船，顶部有一座塔。 2）第二艘船位于图像中央，也停靠在码头。这是一艘较小的船，顶部有一个白色的天篷。 3）第三艘船在图片右侧，停靠在码头。这是一艘较小的船，顶部有一个白色的天篷，与第二艘船类似。 4) 第四艘船位于背景中，位于图像的最右侧。它被第三艘船部分遮挡，但看起来是一艘更大的船，顶部有一座塔。

检索增强生成RAG

RAG

- 一种结合了信息检索技术与语言生成模型的人工智能技术。该技术通过从外部知识库中检索相关信息，并将其作为提示（Prompt）输入给大型语言模型（LLMs），以增强模型处理知识密集型任务的能力，如问答、文本摘要、内容生成等。



检索增强生成RAG

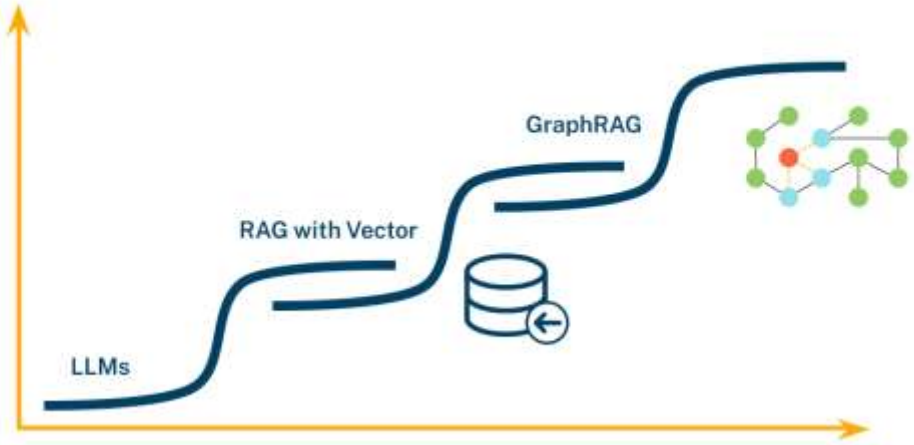


- 文档导入
- 文档标注
- 文档向量化
- 用户提问
- 向量检索
- Reranking重排
- 提示词工程
- 大模型选择

传统RAG的局限

传统RAG存在哪些问题?

- 复杂关系查询： 文本难以表达实体间的多跳、复杂关系
- 事实碎片化： 文本中零散的事实难以有效聚合进行推理。
- 缺乏结构化推理： LLM在纯文本上下文中进行复杂逻辑推理的局限性。
- 可解释性难题： 答案来源难以追溯到具体事实链。

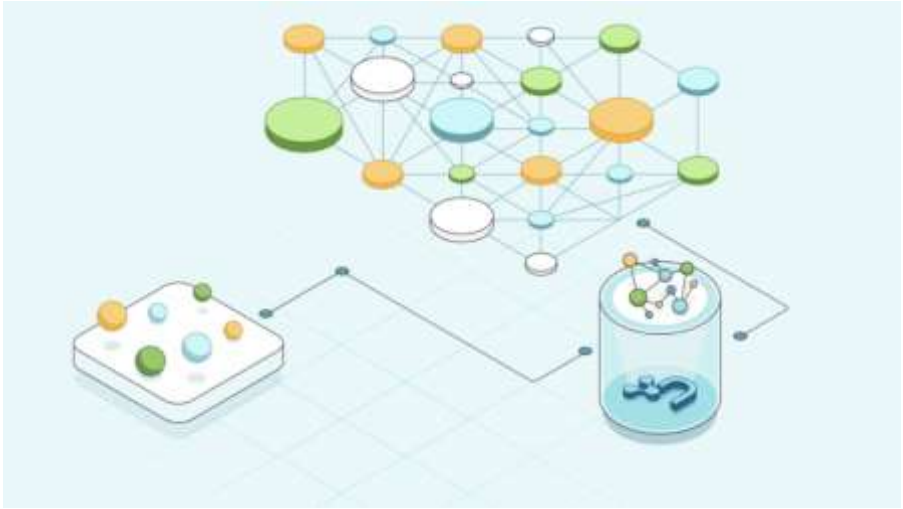


Graph RAG

Graph RAG是什么?

- 基于知识图谱（Knowledge Graphs，KGs）构建。知识图谱是现实世界实体及其关系的结构化表示，主要由两个基本元素组成：
  - 节点（Nodes）： 表示单个实体，如人物、地点、物体或概念。
  - 边（Edges）： 表示节点之间的关系，定义了实体间的连接方式。

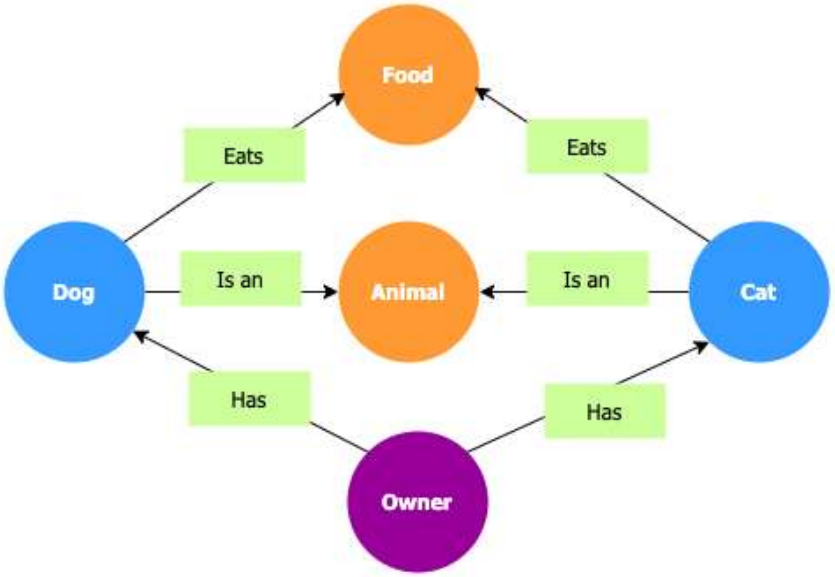




Graph RAG

核心思想

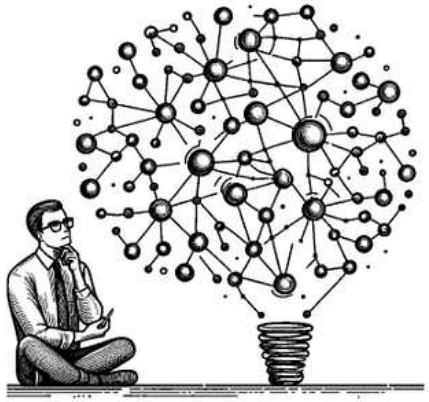
- 知识图谱构建： 将领域知识转化为实体-关系-实体结构。
- 图谱检索： 在知识图谱中查找相关实体、关系和推理路径。
- 图谱到文本转换： 将检索到的图谱信息转化为 LLM 可理解的上下文。
- LLM 增强生成： LLM 基于图谱事实进行深度推理和准确生成。



Agentic RAG

Agentic RAG核心思想

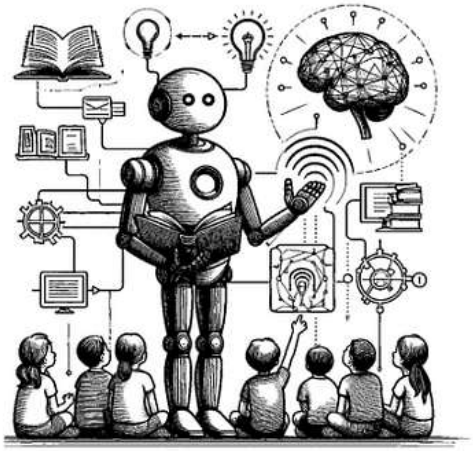
- 将 LLM 提升为拥有规划、工具使用和自我反思能力的智能体(Agent)。它不仅仅是被动地接收上下文，而是能主动决定如何行动，包括何时、何地以及如何检索信息。



LLM



RAG

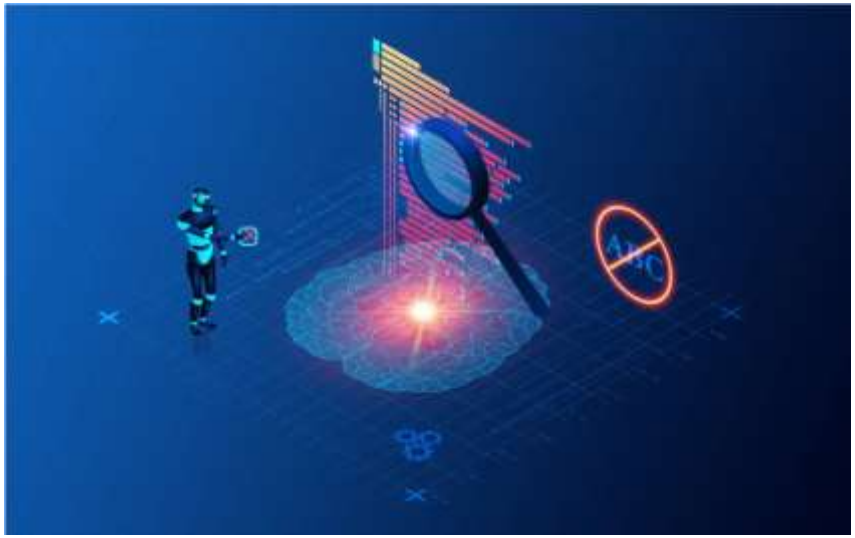


Agentic RAG

Agentic RAG

Agentic RAG核心理念

- 规划 (Planning): 根据复杂任务，将问题分解为子任务，并规划解决步骤。
- 工具使用 (Tool Usage): LLM 不仅能生成文本，还能主动调用外部工具（如搜索引擎、数据库查询、API、甚至代码解释器）。检索器被视为一种特殊的“工具”。
- 自我反思 (Self-Reflection): LLM 能够评估自己的中间结果和最终答案，识别错误并进行修正或重新规划。



小结

- Transformer
- 大语言模型
- Prompt工程
- RAG

End

